POSTS AND TELECOMMUNICATIONS INSTITUTE OF TECHNOLOGY

FACULTY OF INFORMATION TECHNOLOGY 1

Systems Analysis and Design

Topic: Restaurant management

Class: E22HTTT

Members: **Nguyễn Đức Minh (B22DCVT345)**

Customer Searches For Dish Information And Management Staff Views Statistics Of Dishes

DESIGN

Subject No. 23

Duration: 90 minutes

A restaurant management system enables management staff, sale staff and customers to use:

- Management staff: view statistics: dishes, ingredients, customers and suppliers. Manage dish information, make combo menus.
- Warehouse staff: import **materials** from suppliers, manage supplier information
- Sale staff: receive customers, take orders, receive payments at the table, make membership cards for customers, confirm table reservation information and order online.
- Customer: search, book a table and order food online.

Considering two modules and answering questions

- ***Customer searches for dish information***: selects the menu to find dish information → enters the name of the dish to search → the system displays a list of dishes with the name of the keyword entered → clicks on a dish to view details → the system displays detailed information about the dish.
- ***Management staff views statistics of dishes***: selects menu to view statistical reports → **selects dish statistics by revenue** → chooses time to start and end statistics → views dish statistics → selects a **dish to view details** → views the list of order including the dish → selects **view 1 order** → views the selected order.

I. REQUIREMENT

1. CONCEPT EXPLORATION

A. RESTAURANT MANAGEMENT SYSTEM GLOSSARY LIST

NO	English Concept	Vietnamese Concept	Meaning
Concepts related to humans			
1.	Manager	Quản lý nhà hàng	Person who oversees restaurant operations, manages statistics, menus, and staff activities.
2.	Warehouse Staff	Nhân viên kho	Responsible for importing materials from suppliers and managing supplier information.
3.	Sales Staff	Nhân viên phục vụ/thu ngân	Handles customer service: welcoming guests, taking orders, processing payments, managing membership cards.
4.	Customer	Khách hàng	Restaurant visitor who can search for dishes, order food, book tables, and pay.
5.	Chef	Đầu bếp	Prepares dishes based on customer orders and menu.
6.	Supplier	Nhà cung cấp	Provides ingredients, beverages, or other goods for the restaurant.
Concepts related to objects			
7.	Dish	Món ăn	A menu item prepared and served to customers.

8.	Combo Menu	Thực đơn combo	A set of dishes offered together at a special price.
9.	Ingredient	Nguyên liệu	Raw material used for preparing dishes.
10.	Table	Bàn ăn	Customer seating unit, identified for reservations and service.
11.	Membership Card	Thẻ thành viên	A card given to loyal customers with benefits such as discounts or points.
12.	Order	Đơn hàng	A request made by the customer for specific dishes or combos.
13.	Invoice/Bill	Hóa đơn	A financial document listing ordered items and the total payment.
Concepts related to activities			
14.	Reservation	Đặt bàn	Process by which a customer books a table in advance.
15.	Online Order	Đặt món trực tuyến	Customer selects and submits an order via the online system.
16.	Dish Search	Tìm kiếm món ăn	Customer searches for dish details through the menu system.
17.	Import Materials	Nhập nguyên liệu	Warehouse staff registers materials received from suppliers.
18.	Supplier Management	Quản lý nhà cung cấp	Adding, editing, or deleting supplier information.
19.	Statistics	Thống kê	Reports for managers on sales, revenue, and customer trends.
20.	Dish Statistics by Revenue	Thống kê món ăn theo doanh thu	Manager selects time period → system displays revenue per dish.
21.	Customer Information	Thông tin khách hàng	Personal data stored for reservations, memberships, or orders.

22.	Payment Processing	Xử lý thanh toán	Sales staff handles table-side, online, or membership payments.
Concepts related to system rules			
23.	Access Control	Kiểm soát truy cập	Restricts system access based on user role.
24.	User Role	Vai trò người dùng	Defines the set of functions available to each user type.
25.	Report Generation	Tạo báo cáo	System function to automatically generate summaries for managers.
26.	Order Tracking	Theo dõi đơn hàng	Monitor order status from placement to delivery.
27.	Order Cancellation	Hủy đơn hàng	Process where a customer or staff cancels an order.
28.	Reservation Confirmation	Xác nhận đặt bàn	System validates and confirms a table booking.
29.	Revenue Report	Báo cáo doanh thu	Summary of sales revenue within a given period.
30.	Inventory	Kho	The stock of ingredients available in the warehouse.
31.	Inventory Control	Quản lý tồn kho	Tracking and managing ingredient levels to avoid shortages.
32.	Daily Sales Report	Báo cáo doanh số hàng ngày	Summary of sales generated each day.
33.	Customer Feedback	Phản hồi khách hàng	Opinions collected from customers about service and food.
34.	Staff Scheduling	Xếp lịch nhân viên	Assigning shifts and roles to sales staff and chefs.

35.	Food Preparation Time	Thời gian chuẩn bị món ăn	Estimated time required to prepare and serve a dish.
36.	Service Time	Thời gian phục vụ	Time taken to deliver dishes to the customer after ordering.
37.	Table Assignment	Phân công bàn ăn	Allocating a customer to a specific table for dining.

2. BUSINESS MODEL

A. By natural language:

1. Question 1: Objective and scope

Objective: This system is a software application that enables a restaurant to manage its operations including dish management, warehouse materials, sales transactions, reservations, and customer interactions.

Scope:

Type of application: Web-based for customers; desktop/web-based for management staff.

Users in scope: Customer, Management staff, Sale staff, Warehouse staff, Suppliers

Functions in scope:

- Search for dish information.
- View statistics.
- Manage dish information
- Make combo menus
- *Import materials from suppliers*
- *Manage supplier information*
- *Confirm online orders*
- *Confirm table reservation information*
- *Make membership cards for customers*

- *Receive payments*
- *Take orders*
- *Receive customers / assign tables*
- *Order food online*
- *Book a table online*
- *Search for dish information*

2. Users and Functions

Member

- *Login*
- *Logout*

Management Staff

- *View statistics*
    - Dish statistics by revenue
    - Ingredient statistics
    - Customer statistics
    - Supplier statistics
- *Manage dish information* (add/edit/delete)
- *Make combo menus*

Warehouse Staff

- *Import materials from suppliers*
- *Manage supplier information*

Sale Staff

- *Receive customers / assign tables*
- *Take orders*
- *Receive payments*
- *Make membership cards for customers*
- *Confirm table reservation information*
- *Confirm online orders*

Customer

- *Search for dish information*
- *Book a table online*
- *Order food online*

Supplier (Indirect Actor)

- Provides materials for warehouse staff → connected to "Import materials" use case.

3. Detailed Business Process of Functions

Module 1: Customer searches for dish information

- Customer selects Menu → Dish Information.
- Customer enters the dish name (keyword) into the search bar.
    - If no dishes match the selected dish→ repeat from entering data step
- The system searches and returns a list of matching dishes.
- Customer clicks on one dish.
- The system displays detailed dish information: dish name, category, ingredients, description, price, and availability.

Module 2: Management staff views statistics of dishes

- Management staff logs in.
- Selects Menu → Statistical Reports → Dish statistics by revenue.
- Enters start date and end date for the report.
    - If the period invalid (start date is after end date, missing start date or end date, selected date range is beyond today's date) → repeat from entering data step
    - If the period is valid → next step
- The system retrieves and displays dish statistics (sorted by total revenue).
- Manager clicks on a dish → The system shows the list of orders including that dish.
    - If no dishes match the selected time period → repeat from entering data step

    - If there are dishes that match the selected time period → next step
- Manager selects one order → The system displays detailed order information: order ID, customer info, ordered dishes, quantities, total price.

4. Information about Related Objects

   Group of information related to people:

- User: name, account, password, date of birth, address, phone number, role.
- Customer: same as user, plus membership card ID.
- Employee: same as user, plus position.
- Manager: same as employee.
- Sales staff: same as employee.
- Warehouse staff: same as employee.
- Supplier: ID, name, address, phone number.

   Group of information related to objects:

- Table: table name, number of seats.
- Dish: name, ingredients, price, description.
- Combo: name, list of dishes in the combo, price, description.
- Ingredient: name, unit, unit price.
- Invoice: creation date, total amount.

   Group of information related to statistics:

- Statistics of dishes by revenue.
- Statistics of customers by revenue.
- Statistics of suppliers by quantity of goods imported.

5. Relation among Objects

- A customer can reserve tables many times. One reservation can include many tables. A table can be reserved by one customer at a time and by many customers at different times.
- A customer can order many dishes, and a dish can be ordered by many customers.

- A customer can order many combos, and a combo can be ordered by many customers.
- A combo can contain many dishes, and a dish can belong to many combos.
- A dish can have many ingredients, and an ingredient can belong to many dishes.
- One ingredient import session can include many ingredients, and an ingredient can be imported many times.
- An employee can process many invoices, but an invoice can only be processed by one employee.
- A customer can have many invoices, but an invoice belongs to only one customer.
- A supplier can have many ingredient import invoices, but an ingredient import invoice belongs to only one supplier.

B. By UML:

Actors:

- Direct actors: management staff, sale staff, warehouse staff, customer
- Indirect actors: suppliers.

General use case diagram (for the whole system):

Member

- *Login*
- *Logout*

Management Staff

- *View statistics*
  - Dish statistics by revenue
  - Ingredient statistics
  - Customer statistics
  - Supplier statistics
- *Manage dish information* (add/edit/delete)
- *Make combo menus*

Warehouse Staff

- *Import materials from suppliers*
- *Manage supplier information*

Sale Staff

- *Receive customers / assign tables*
- *Take orders*
- *Receive payments*
- *Make membership cards for customers*
- *Confirm table reservation information*
- *Confirm online orders*

Customer

- *Search for dish information*
- *Book a table online*
- *Order food online*

Supplier (Indirect Actor)

- Provides materials for warehouse staff → connected to "Import materials" use case.

Use case descriptions:

+ Book a table online: This UC allows Customer to book a table online.
+ Order food online: This UC allows Customer to order food through online system.
+ Search dish information: This UC allows Customer to search for dish information.

+ Manage dish information: This UC allows Management staff add/edit/delete dishes.
+ Make combo menus: This UC allows Management staff to make combos menus.
+ View statistics: This UC allows Management staff to view all type of statistic.

+ Manage supplier: This UC allows Warehouse staff to view, add or delete suppliers.
+ Import materials: This UC allows Warehouse staff to import materials.

+ Make membership card: This UC allows Sale staff to add new membership card.

+ Receive payments: This UC allows Sale staff to accept payments.

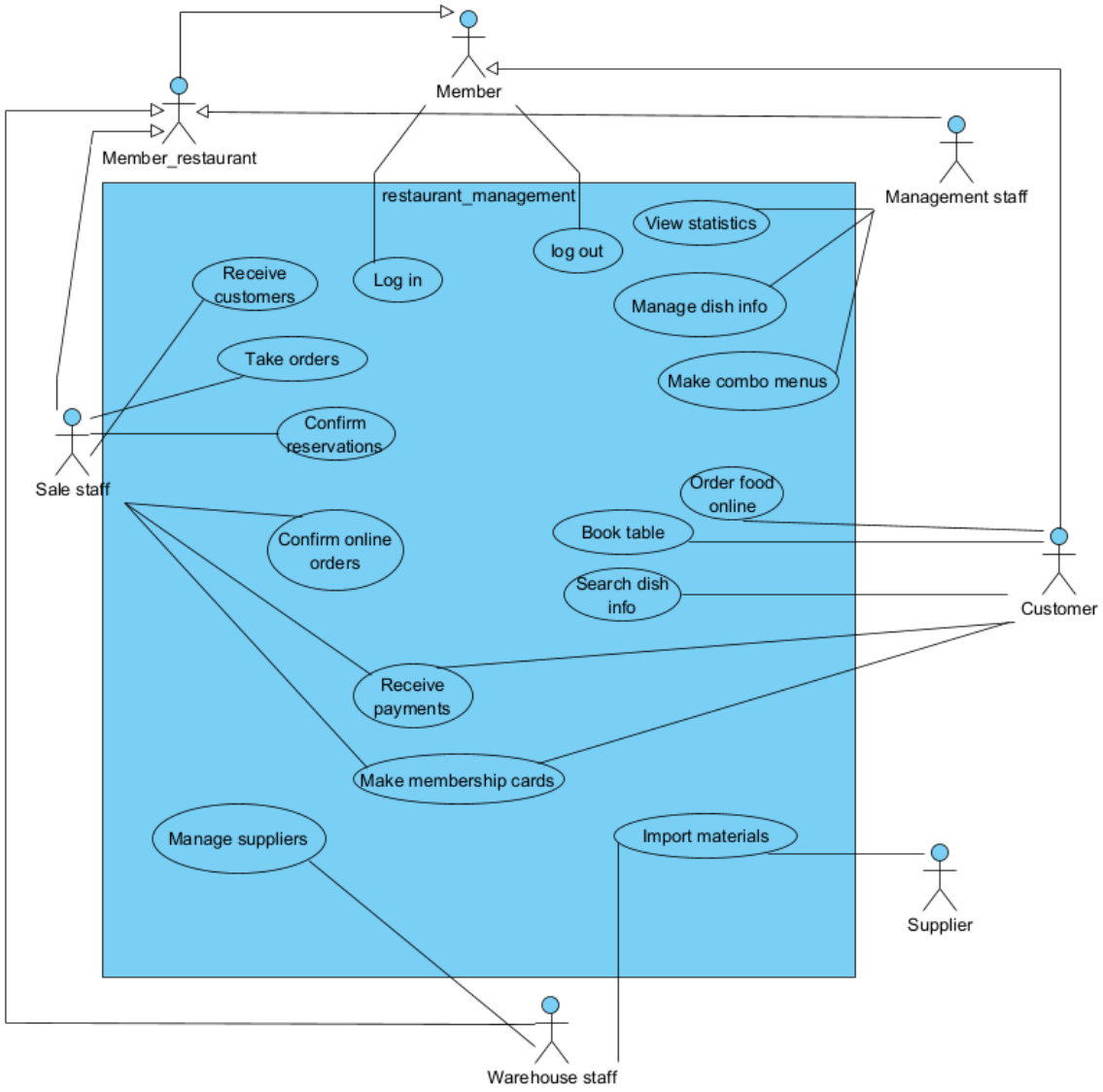
+ Receive customers / assign tables: This UC allows Sale staff to receive customers and assign tables.

+ Take orders: This UC allows Sale staff to take orders.

+ Confirm table reservation information: This UC allows Sale staff to confirm table reservation information

+ Confirm online orders: This UC allows Sale staff to confirm online orders

The obtained general use case diagram is:



Detail use case diagram:

UC1: ***Customer searches for dish information***

This Usecase require customer to login

1. Usecase description:

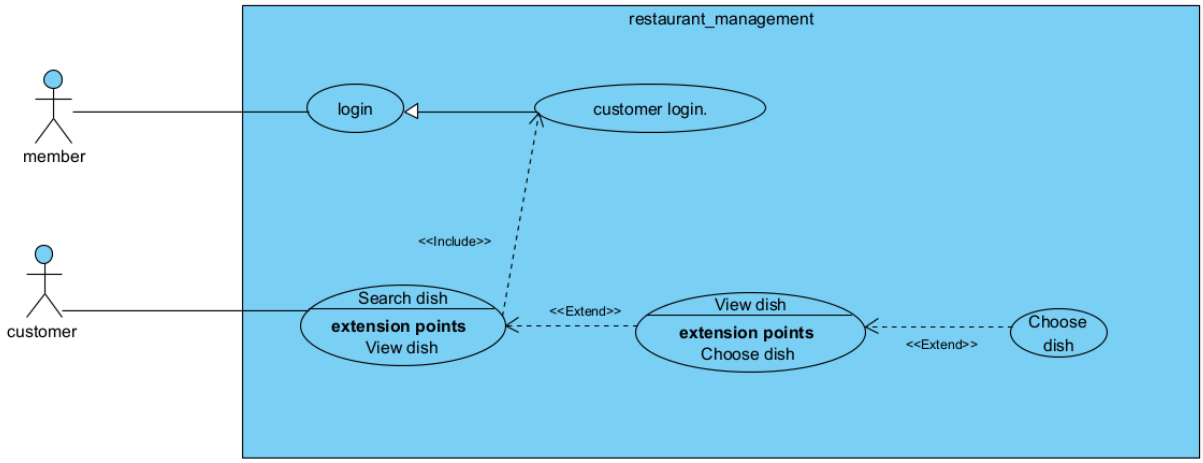
   Customer Login: This UC allows customer to login

   Search dish: This UC allows customer to search for dish

   View dish: This UC allows customer to view dish information.

   Choose dish: This UC allows customer to choose dish.

2. Detailed Usecase diagram:



UC2: ***Management staff views statistics of dishes***

This Usecase require Management staff to login

1. Use case description:

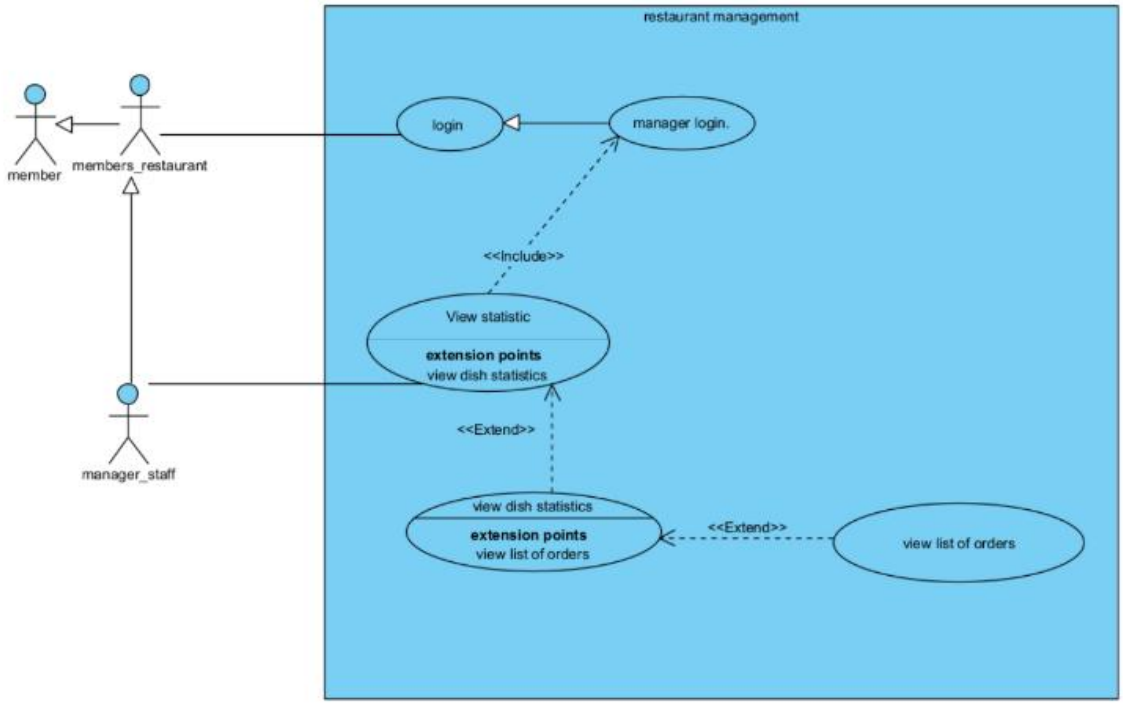
   Manager Login: This UC allows Management staff to login

   View statistic: This UC allows Management staff to view orders and dishes statistics

   View dish statistics: This UC allows Management staff to view the list of dishes

   View list of orders: This UC allows Management staff to view list of orders

2. Detailed Usecase diagram:



II. ANALYSIS

Scenario

a) Scenario for module “Search dish information”.

Use case	Search dish information
Actor	Customer
Precondition	Customer login successfully and proceed to search for dish info
Postcondition	The customer can view a list of dishes containing the search keyword and may also view the details of a selected dish.
Main scenario	1. After logging in, Customer A selects the function to search for dish information. 2. The dish information search interface appears, consisting of a text field and a search button search

<table>
<tr>
<td></td>
<td>
3. Customer A enters the word “Chả” and presses the search button.
4. The interface displays a list of dishes containing the entered keyword.

Chả search

<table>
<tr><th>TT</th><th>Name dish</th><th>Price</th><th>View detail</th></tr>
<tr><td>1</td><td>Bánh cuốn chả lụa</td><td>40000</td><td>click</td></tr>
<tr><td>2</td><td>Chả lụa chiên</td><td>30000</td><td>click</td></tr>
<tr><td>3</td><td>Chả lụa xốt cà chua</td><td>35000</td><td>click</td></tr>
</table>
5. Customer A clicks on the first item in the displayed list.
6. The interface displays the detailed information of the dish “Bánh cuốn chả lụa”.
Return

Detail about dish “Bánh cuốn chả lụa”
Name dish: Bánh cuốn chả lụa
Price: 40000
Ingredients: bánh, chả
Description: Bánh cuốn với chả lụa
7. Customer A clicks the back button.
8. The interface from Step 4 is displayed again.
</td>
</tr>
<tr>
<td>Exception</td>
<td>
3. Customer does not enter any input and presses the search button.
3.1. The interface displays the complete list of dishes available in the restaurant..
4. No dish matches the entered keyword
4.1 The customer re-enters a new keyword and presses the search button
</td>
</tr>
</table>

b) Scenario for module “View Statistic of dishes”.

<table>
<tr><td>Use case</td><td>View Statistic of dishes</td></tr>
<tr><td>Actor</td><td>Management staff</td></tr>
<tr><td>Precondition</td><td>The management staff has successfully logged in, and the system already contains dishes data and orders data</td></tr>
<tr><td>Postcondition</td><td>Management staff obtains statistical reports of dishes for the specified time period.</td></tr>
<tr><td>Main scenario</td><td>1. From the main interface, the management staff selects the menu option “View Statistical Reports.”.
2. The system displays the statistical report interface, with available categories: Dish Statistics by Revenue, Customer Statistics by Revenue, Ingredients Statistics by Revenue, Suppliers Statistics by Revenue.
3. The management staff selects “Dish Statistics by Revenue.”
4. The system prompts for the start date and end date of the statistics.
5. The management staff enters the start and end date and confirms

<table><tr><td>Start date</td><td>End date</td></tr><tr><td>01-01-2023</td><td>01-01-2024</td></tr></table>
6. The system generates and displays dish statistics for the selected period.

<table><tr><td>Dish ID</td><td>Name Dish</td><td>Price</td><td>Revenue</td><td>Details</td></tr></table></td></tr>
</table>

	1	Bánh cuốn chả lụa	40000	80000	click
	2	Chả lụa chiên	30000	40000	click

7. Management staff select 1 line Bánh cuốn chả lụa on the display list
8. The system displays a detailed view of the selected dish, including order count, total revenue, and list of orders containing this dish:

   order count: 1

   Revenue: 80000

Order_ID
1

9. Management staff choose the first order
10. Interface for detailed order appear:

Order ID: 1

Customer Name: M

Table Number: 1

Order Date/Time: 01-02-2023

Ordered Dishes: Bánh cuốn chả lụa

Total Amount: 80000

Exception	

	5. Invalid or empty date input → The system displays a message *"Please enter a valid start and end date."*. 6. No dishes found within the selected period → The system displays a message *"No dish statistics available for the chosen time range."* 8. The selected dish has no related orders in the period → The system shows *"No orders found for this dish in the selected period."* 9. If the management staff clicks on a dish but does not select an order → The system remains on the dish details screen.

Entity class diagram:

1. Describe the modules within a paragraph:

A restaurant management system (RestMan) allows managers, sales staff, and customers to use the system. After logging in, each actor can perform corresponding tasks. Manager: View various statistics on dishes, ingredients, customers, and suppliers. Manage dish information and create combo menus. Warehouse staff: Import materials from suppliers and manage supplier information. Sales staff: Receive customers, take orders, handle payments at tables, issue membership cards for customers, and confirm customer reservations and online orders. Customer: Search for dishes, book tables, and place online orders. Customer function – Search for dish information: Select the menu option to search for dishes → enter the dish's name to search → the system displays a list of dishes containing the entered keyword → click on a dish to view details → the system shows detailed information about the dish. Staff function – Statistics on dishes by revenue: Select the menu option to view reports → choose "statistics on dishes by revenue" → select start and end time for the report → view dish statistics → select a dish

to view details → see all instances where the dish was ordered → select one instance → view the corresponding invoice.

2. Extract nouns:

- People-related nouns: Manager, sales staff, warehouse staff, customer, supplier.
- Object-related nouns: Dish, restaurant, material, dish combo, membership card, table, invoice, food.
- Information-related nouns: System, statistical report, menu, information, function, list, name, keyword, result, quantity, date and time, unit price, amount, total amount.

3. Classify nouns:

Abstract nouns: system, statistical report, information, function, list → exclude
People-related nouns:

- User -> class User: name, username, password, date of birth, address, telephone
- Staff -> class Staff: inherits from User,
  add attribute: position.
- Customer → class Customer inherits from User:
  add attributes: customer ID, membership card
- Supplier → class Supplier: supplier ID, name, address, telephone

Object-related nouns:

- Food class OrderFood: OrderFood ID.
- Dish → class Dish: dish ID, name, description, price, category
- Combo → class Combo: combo ID, name, price, description
- Table → class Table: table ID, name, description, number of seats
- Bill invoice → class InvoiceBill: total amount, creation date
- Supply invoice → class InvoiceSupplies: total amount, creation date
- Material→ class Material: Material ID, name, note

4. Quantity relationships among classes:

1. User – Customer

- A User can be a Customer , One Customer has exactly one AccountCustomer. (1 →1)

2. User – Staff

- A User can also be a Staff, One Staff can be responsible for issuing many InvoiceBill or InvoiceSupplies.

3. Customer – InvoiceBill

- One Customer can have many InvoiceBill, Each InvoiceBill belongs to exactly one Customer.

  (1 → n)

4. Supplier – InvoiceSupplies

- One Supplier can generate many InvoiceSupplies, Each InvoiceSupplies is associated with exactly one Supplier.

  (1 → n)

5. InvoiceSupplies – Material

- One InvoiceSupplies can include many Material, One Material can appear in many InvoiceSupplies.

  (n → n via ImportedMaterial)

6. Table – TableStat – BookedTable

- One Table can have many TableStat records over time, Each TableStat is linked to exactly one Table, A Table can be reserved via BookedTable.

7. OrderFood– InvoiceBill

- Each OrderFood can generate one InvoiceBill, Each InvoiceBill is tied to one OrderFood.

  (1 → 1)

8. Combo – ComboDish – Dish

- One Combo consists of many Dish, One Dish can belong to many Combo. (n → n through ComboDish)

9. OrderFood – OrderedDish – Dish

- One OrderFood can include many OrderedDish, Each OrderedDish belongs to exactly one Dish, One Dish can appear in many OrderedDish. (n → n through OrderedDish)

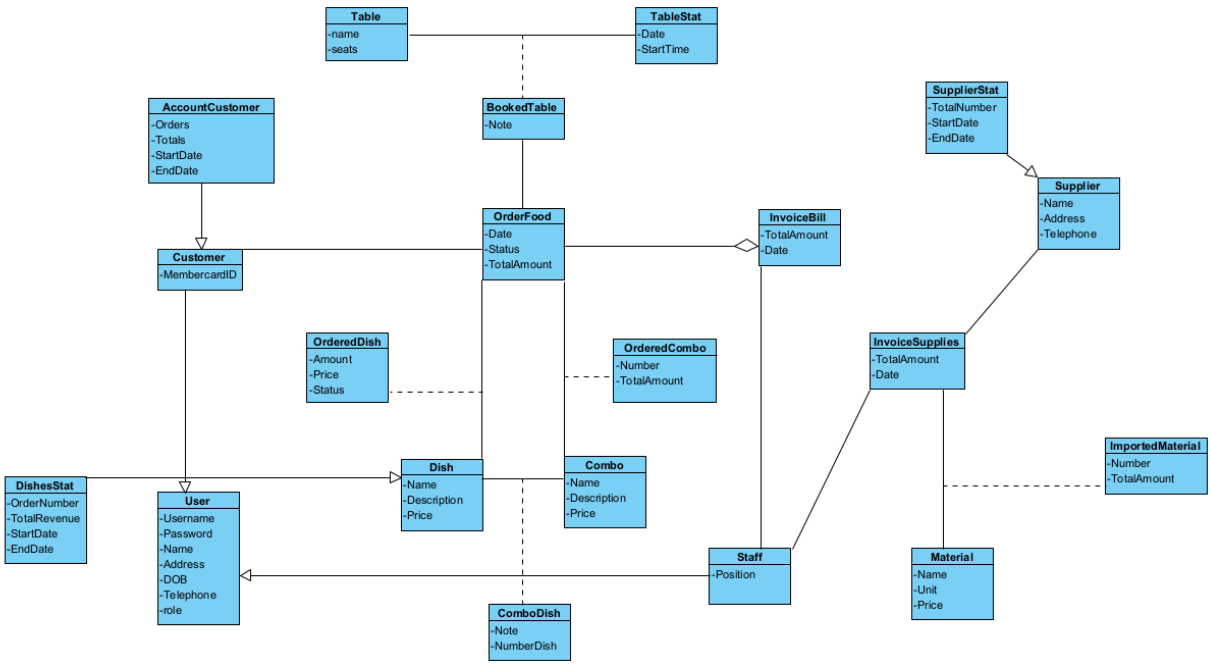
10. OrderFood – OrderedCombo – Combo

- One OrderFood can include many OrderedCombo, Each OrderedCombo belongs to exactly one Combo, One Combo can appear in many OrderedCombo. (n → n through OrderedCombo)

5. Object relationships among classes

- Customer is a specialization of User.
- Staff is a specialization of User.
- Customer is linked to AccountCustomer (1 → 1).
- Customer can make many OrderFood, each OrderFood belongs to exactly one Customer.
- OrderFood can generate one InvoiceBill, each InvoiceBill belongs to exactly one OrderFood.
- Staff can issue many InvoiceBill and InvoiceSupplies, each bill is tied to exactly one Staff.
- Supplier can generate many InvoiceSupplies, each InvoiceSupplies belongs to exactly one Supplier.
- InvoiceSupplies and Material are linked by ImportedMaterial (many-to-many).
- Supplier is linked with SupplierStat (1 → 1).
- Customer is linked with DishesStat (statistics by order, revenue, and time).
- Table can have many TableStat, each TableStat belongs to exactly one Table.

- Table can be reserved through BookedTable (1 → n).
- OrderFood is connected to TableStat (an order is tied to a specific table usage).
- OrderFood can include many OrderedDish, each OrderedDish belongs to exactly one Dish.
- OrderFood can include many OrderedCombo, each OrderedCombo belongs to exactly one Combo.
- Dish and Combo are linked by ComboDish (many-to-many).



Extract the boundary and control classes.

a) Static Analysis of Module "Search Dish Information"

- Initially, the customer's main interface → propose class MainView, which needs the following component:

- Search for dish information: submit type

- Interface for searching dish information → propose class SearchDishView, which needs the following components:

- Enter name: input

- Search button: submit type
- Dish list table: output, submit

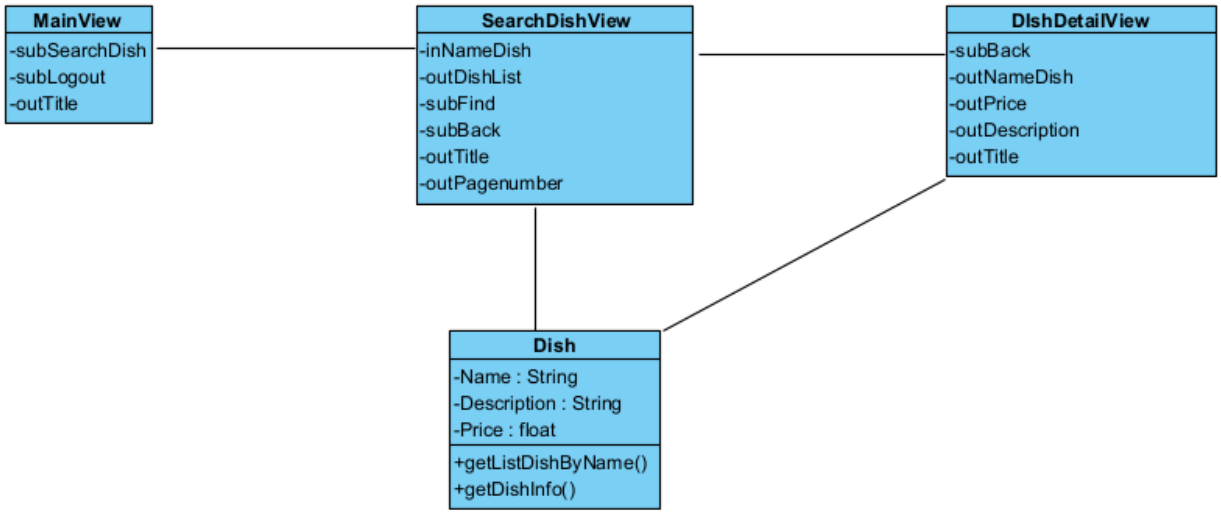
- To obtain the dish list, processing is needed in the system:
  - Input: dish's name
  - Output: list of dishes whose names contain the keyword

    → Proposed method getListDishByName(), assigned to class Dish

- Interface for viewing dish details → propose class DishDetailView, which needs the following components:
  - Dish details: output
  - Back button: submit type

- To obtain detailed dish information, processing is needed in the system:
  - Input: Dish
  - Output: detailed information about the dish

    → Proposed method getDishInfo() assigned to class Dish



b) Static Analysis of Module "View Statistic of dishes"

- Initially, the customer's main interface → propose class MainView, which needs the following component:

- Choose StatView: submit type

- Interface for selecting type of statistic → propose class StatView, which needs the following component:

- List of statistic types: both input and output
- List of Dish: both input and output
- View button: submit type

- To obtain the list of Dish, processing is needed in the system:

- Retrieve list of Dishes
- Input: StartDate and EndDate
- Output: all Dishes

  →Proposed method getDishStat(), assigned to class DishStat

- Interface for statistics by Order → propose class OrderListView, which needs the following component:

- List of orders: output, submit.

- To obtain the list of Orders, processing is needed in the system:

- Retrieve Order statistics for the selected Dish
- Input: Dish
- Output: list of orders

  →Proposed method getListOrderByDish(), assigned to class OrderFood

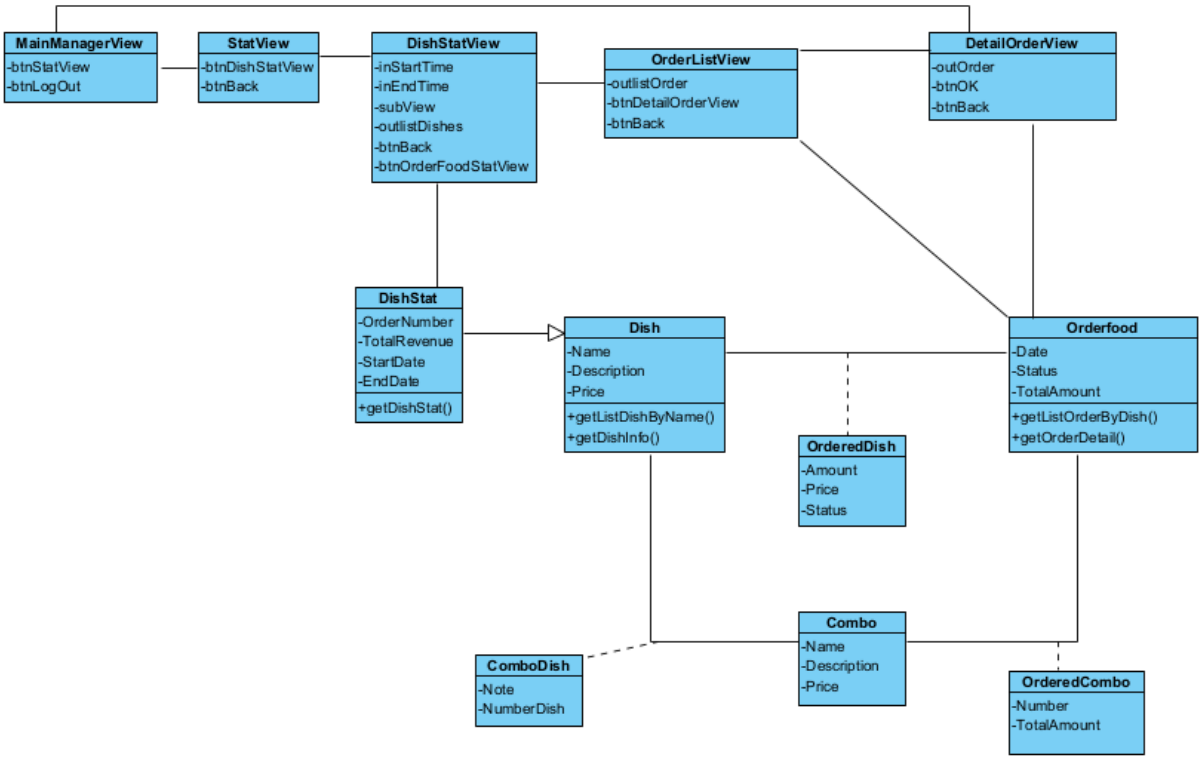
- Interface for detailed of an Order→ propose class DetailOrderView, which requires:

- Order detail: output

- To obtain detailed order, processing is needed in the system:

- Retrieve Order details for the selected Order
- Input: Order
- Output: Order details

  →Proposed method getOrderDetail(), assigned to class OrderFood

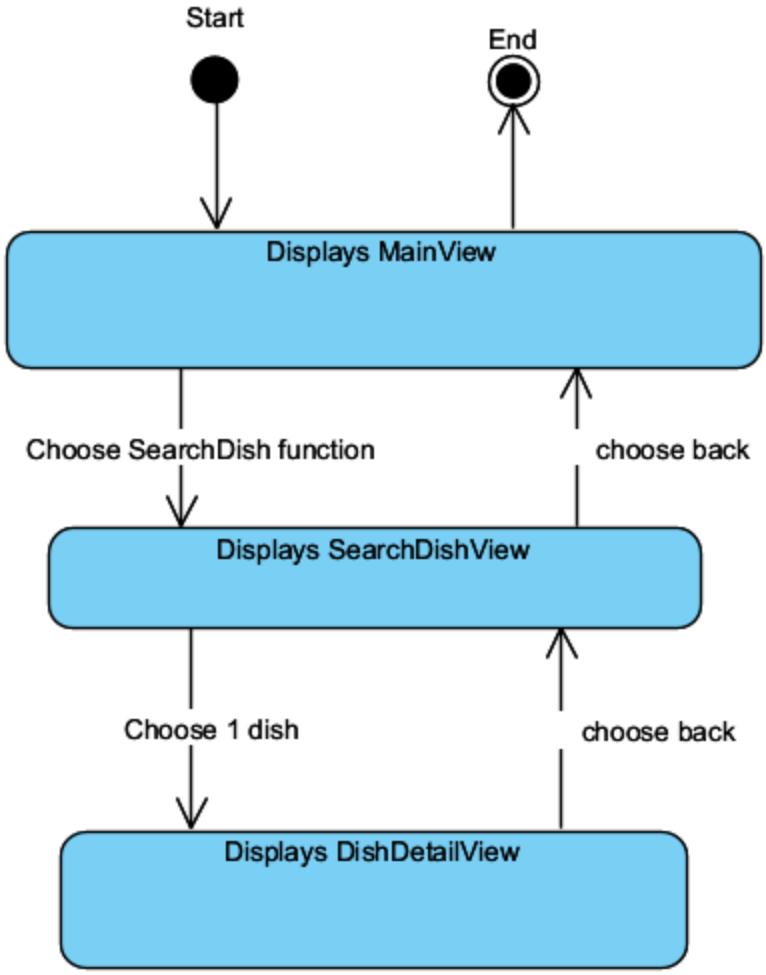


Activity analysing

a) Module Search Dish:

State Diagram:

- From the customer's main interface, if the Search Dish function is selected, the system navigates to the Search Dish interface.
- In the Search Dish interface, if the customer enters a dish's name and presses Search, the interface will display a list of dishes whose names contain the entered keyword.
- In the Search Dish interface, if the customer selects a dish from the displayed list, the system navigates to the Dish Detail interface.

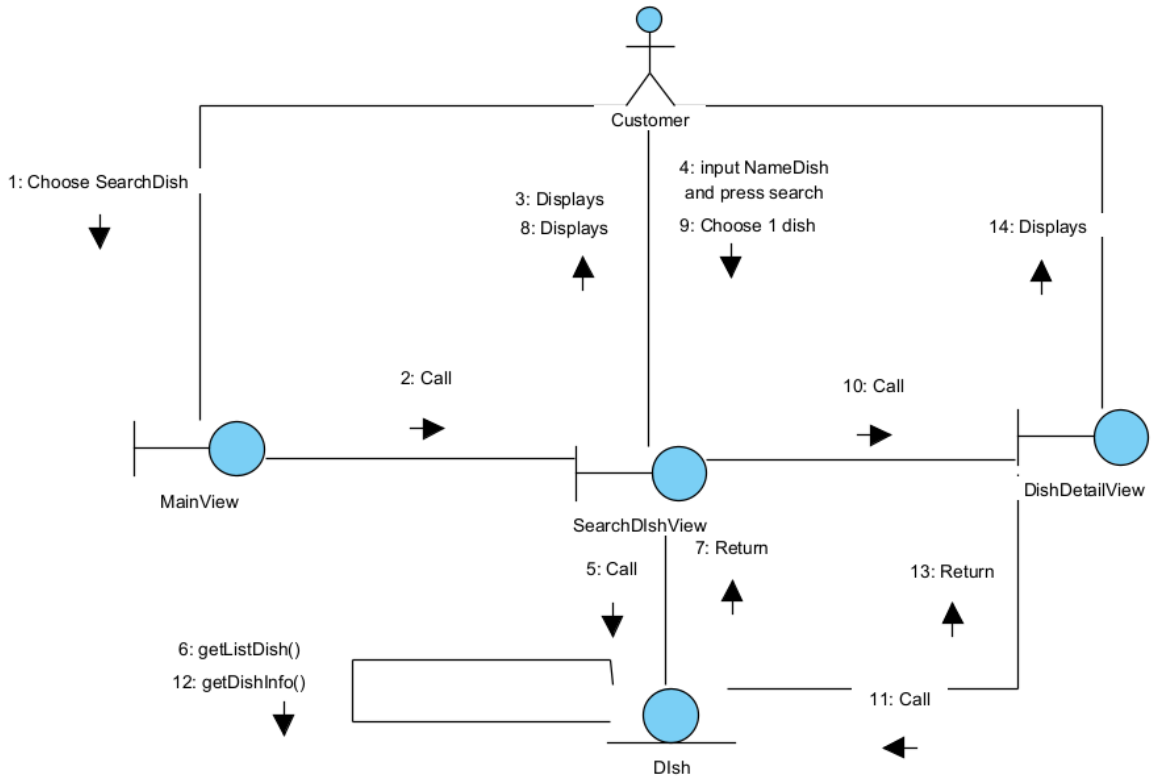


Scenario v.2:

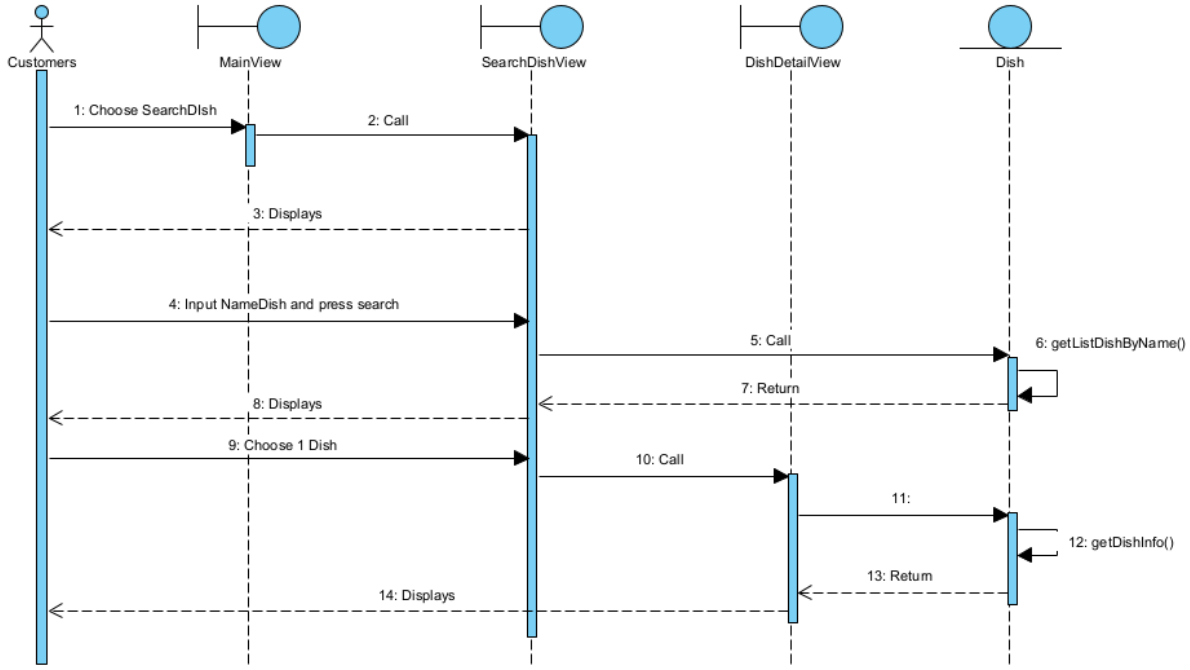
1. In the customer's main interface, after logging in, the customer selects the function Search Dish Information.
2. Class MainView calls SearchDishView.
3. Class SearchDishView is displayed to the customer.
4. The customer enters the name of the dish to search.
5. Class SearchDishView calls class Dish to request dishes whose names contain the entered keyword.
6. Class Dish searches for dishes that satisfy the request.
7. Class Dish returns the result to class SearchDishView.
8. Class SearchDishView displays the result to the customer.
9. The customer selects one dish.

10. SearchDishView calls class DishDetailView.
11. Class DishDetailView calls class Dish to request information about the selected dish.
12. Class Dish retrieves the dish information.
13. Class Dish returns the result to class DishDetailView.
14. Class DishDetailView displays the detailed information to the customer.

Communication Diagram:



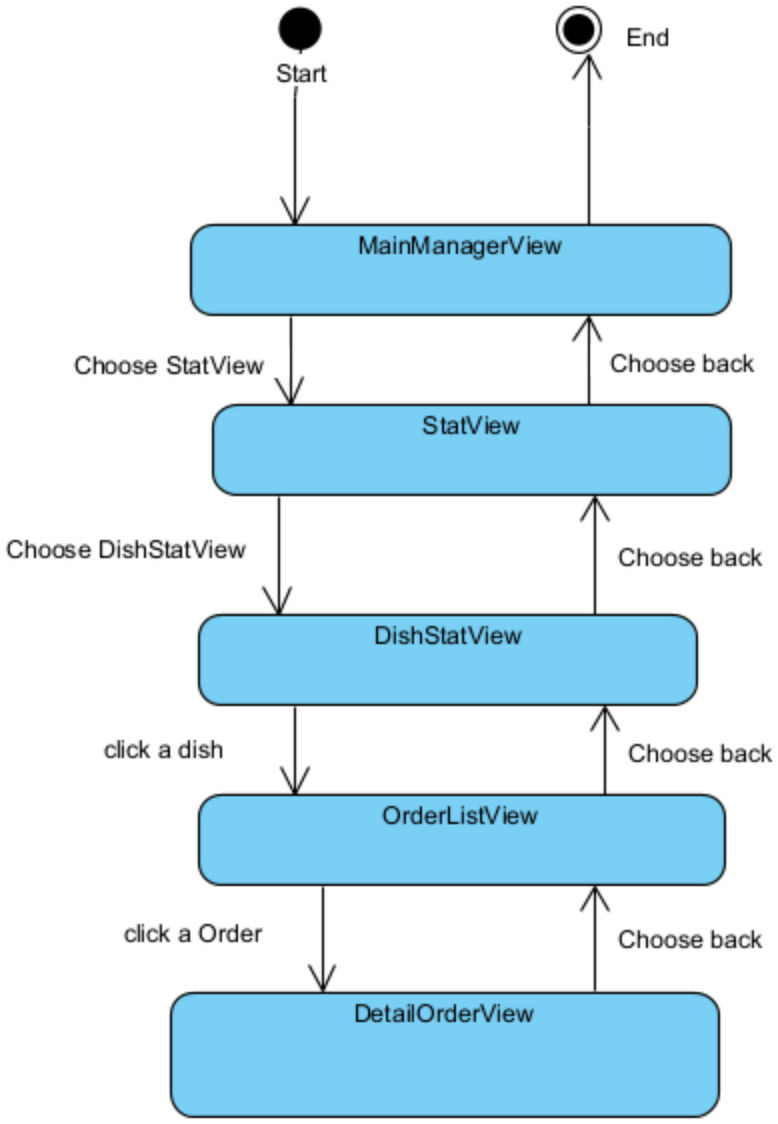
Sequence Diagram:



b) Module View Dish Statistic:

State Diagram:

- From the manager's main interface (MainManagerView), if the StatView option is selected, the system navigates to the StatView interface.
- In the StatView interface, if the DishStatView option is selected, the system navigates to the DishStatView interface.
- In the DishStatView interface, if the manager clicks on a dish, the system navigates to the OrderListView interface, displaying statistics of orders containing that dish.
- In the OrderListView interface, if the manager clicks on an order, the system navigates to the DetailOrderView interface, showing full details of that order.
- In every interface, if the Back option is chosen, the system returns to the previous interface.
- From MainManagerView, choosing Back exits the system.

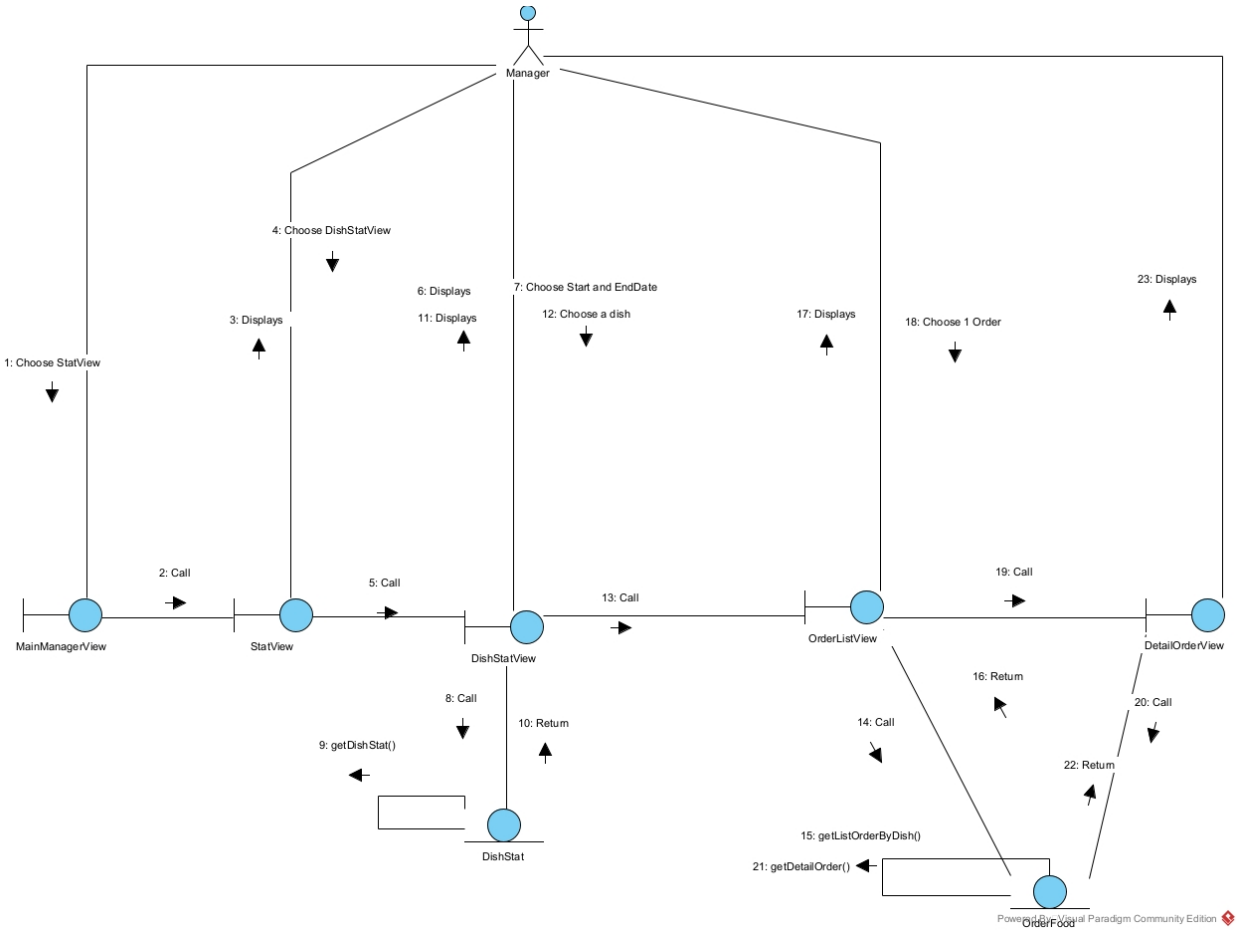


Scenario v.2:

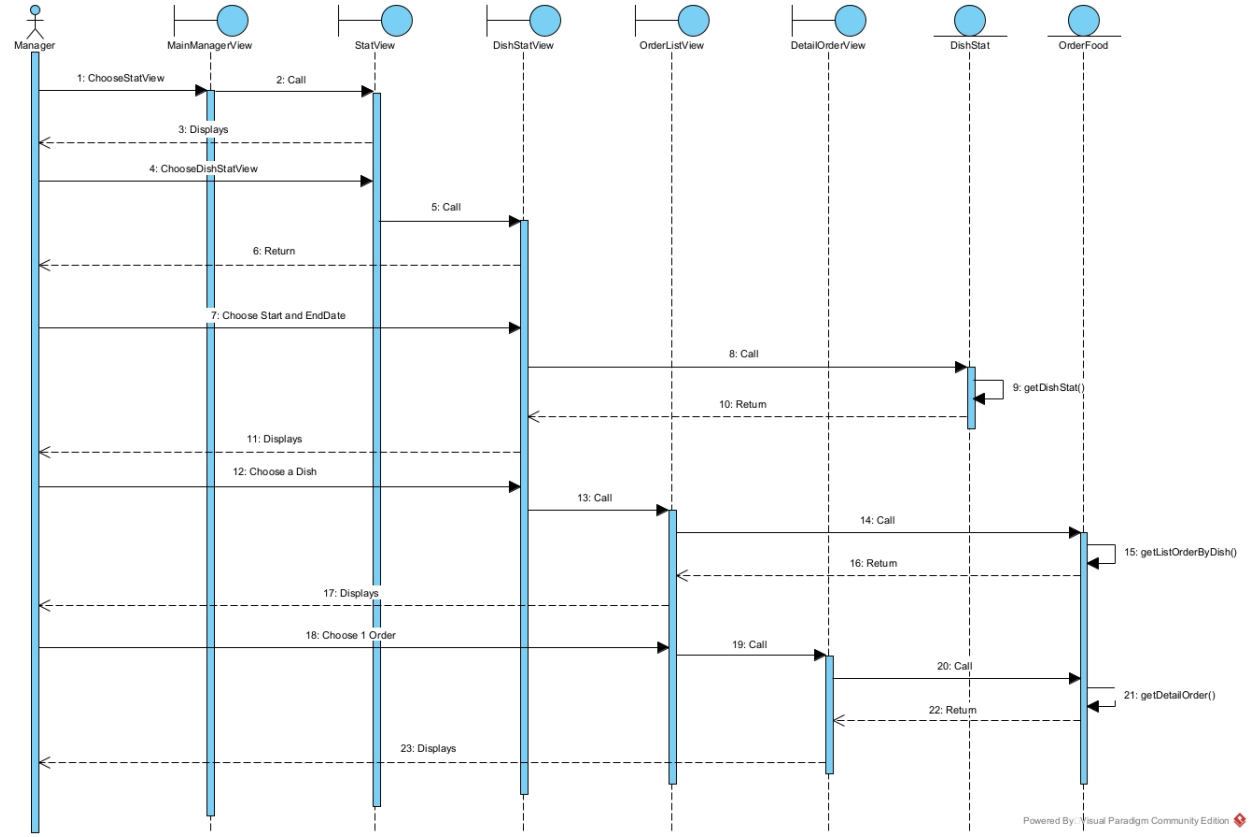
1. On the Main Manager interface, the manager chooses View Statistics.
2. The Main ManagerView calls the StatView interface.
3. The StatView interface is displayed to the manager.
4. The manager chooses View Dish Statistics.
5. The StatView calls the DishStatView interface.
6. The DishStatView interface is displayed to the manager.
7. The manager selects a start date and end date.

8. The DishStatView calls the DishStat class to request dish statistics.
9. The DishStat class executes getDishStat().
10. The DishStat returns the statistical data.
11. The DishStatView interface displays the data to the manager.
12. The manager clicks on a dish.
13. The DishStatView calls the OrderListView interface.
14. The OrderListView calls the OrderFood class.
15. The OrderFood class executes getListOrderByDish().
16. The OrderFood returns the result.
17. The OrderListView interface displays the order list to the manager.
18. The manager selects one order.
19. The OrderListView calls the DetailOrderView interface.
20. The DetailOrderView calls the OrderFood class.
21. The OrderFood class executes getDetailOrder().
22. The OrderFood returns the order details.
23. The DetailOrderView interface displays the details of the selected order to the manager.

Communication Diagram:



Sequence Diagram:



III. Design

Entity Class Design

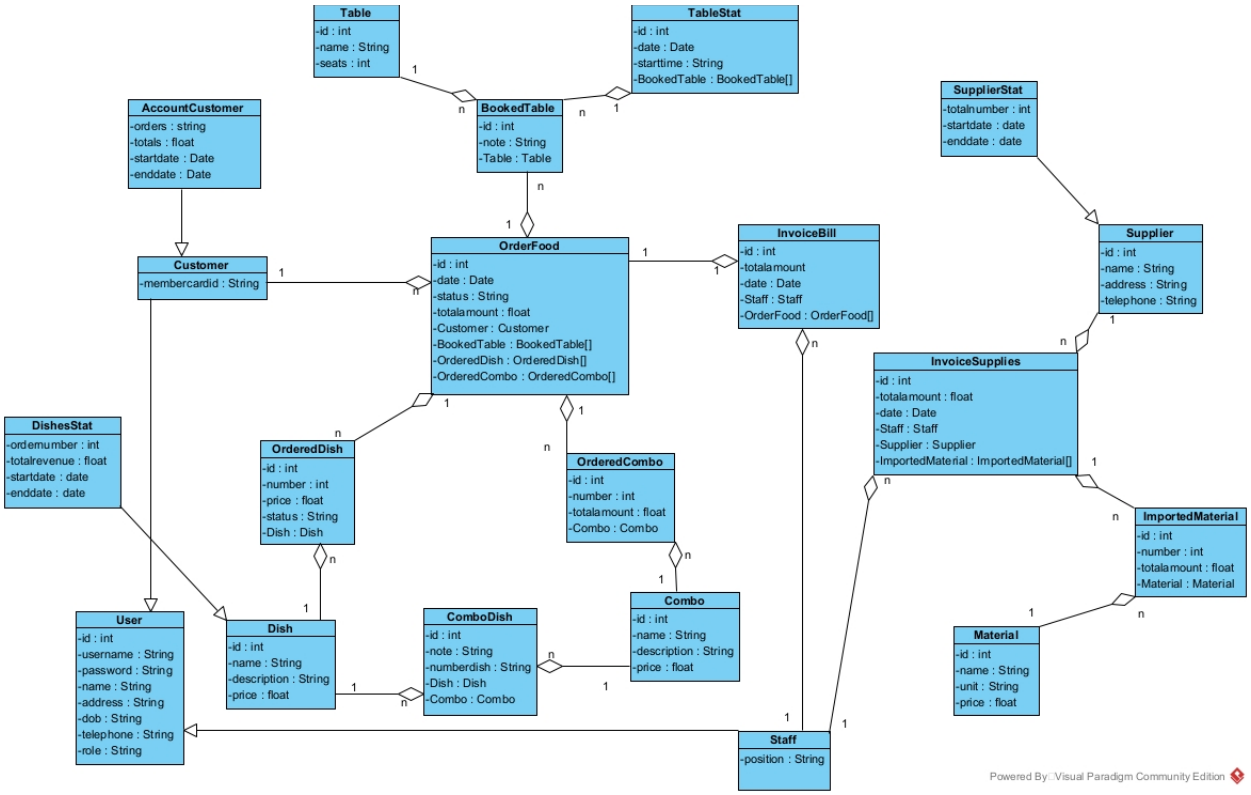
Step 1: Classes are supplemented with id attribute: except for statistical classes, Staff class, Customer class

Step 2: Class attributes are supplemented with attributes according to Java programming language data types

Step 3: Many-to-many relationship handling:

- Combo – Dish relationship → ComboDish converts to ComboDish containing Combo and Dish

- OrderFood – Dish relationship → OrderedDish converts to OrderedDish containing Dish, OrderFood containing OrderedDish
- OrderFood – Combo relationship → OrderedCombo converts to OrderedCombo containing Combo, OrderFood containing OrderedCombo
- OrderFood – Table relationship → BookedTable converts to BookedTable containing Table, OrderFood containing BookedTable
- InvoiceSupplies – Material relationship → ImportedMaterial converts to ImportedMaterial containing Material, InvoiceSupplies containing ImportedMaterial



Database Design:

Step 1: Each entity class proposes corresponding table:

- User class → tblUser table
- Customer class → tblCustomer table

- Staff class → tblStaff table
- OrderFood class → tblOrderFood table
- Table class → tblTable table
- BookedTable class → tblBookedTable table
- Dish class → tblDish table
- Combo class → tblCombo table
- ComboDish class → tblComboDish table
- OrderedDish class → tblOrderedDish table
- OrderedCombo class → tblOrderedCombo table
- InvoiceBill class → tblInvoiceBill table
- Supplier class → tblSupplier table
- InvoiceSupplies class → tblInvoiceSupplies table
- Material class → tblMaterial table
- ImportedMaterial class → tblImportedMaterial table

Step 2: Convert non-object attributes of entity classes to corresponding table attributes:

- tblUser has attributes: id, username, password, name, address, dob, telephone, role
- tblCustomer has attributes: id, membercardid
- tblStaff has attributes: id, position
- tblOrderFood has attributes: id, date, status, totalamount
- tblTable has attributes: id, name, seats
- tblBookedTable has attributes: id, note
- tblDish has attributes: id, name, description, price
- tblCombo has attributes: id, name, description, price
- tblComboDish has attributes: id, note, numberdish
- tblOrderedDish has attributes: id, number, price, status
- tblOrderedCombo has attributes: id, number, totalamount, status
- tblInvoiceBill has attributes: id, totalamount, date
- tblSupplier has attributes: id, name, address, telephone

- tblInvoiceSupplies has attributes: id, totalamount, date
- tblMaterial has attributes: id, name, unit, price
- tblImportedMaterial has attributes: id, number, totalamount

Step 3: Convert cardinality relationships between entity classes to cardinality relationships between tables:

- 1 tblUser - 1 tblCustomer
- 1 tblUser - 1 tblStaff
- 1 tblCustomer – n tblOrderFood
- 1 tblOrderFood – n tblBookedTable
- 1 tblTable – n tblBookedTable
- 1 tblOrderFood – n tblOrderedDish
- 1 tblDish – n tblOrderedDish
- 1 tblOrderFood – n tblOrderedCombo
- 1 tblCombo – n tblOrderedCombo
- 1 tblCombo – n tblComboDish
- 1 tblDish – n tblComboDish
- 1 tblOrderFood – 1 tblInvoiceBill
- 1 tblStaff – n tblInvoiceBill
- 1 tblStaff – n tblInvoiceSupplies
- 1 tblSupplier – n tblInvoiceSupplies
- 1 tblInvoiceSupplies – n tblImportedMaterial
- 1 tblMaterial – n tblImportedMaterial

Step 4: Add key attributes. Primary keys are established with the id attribute of corresponding tables, except for tables: tblCustomer, tblStaff.

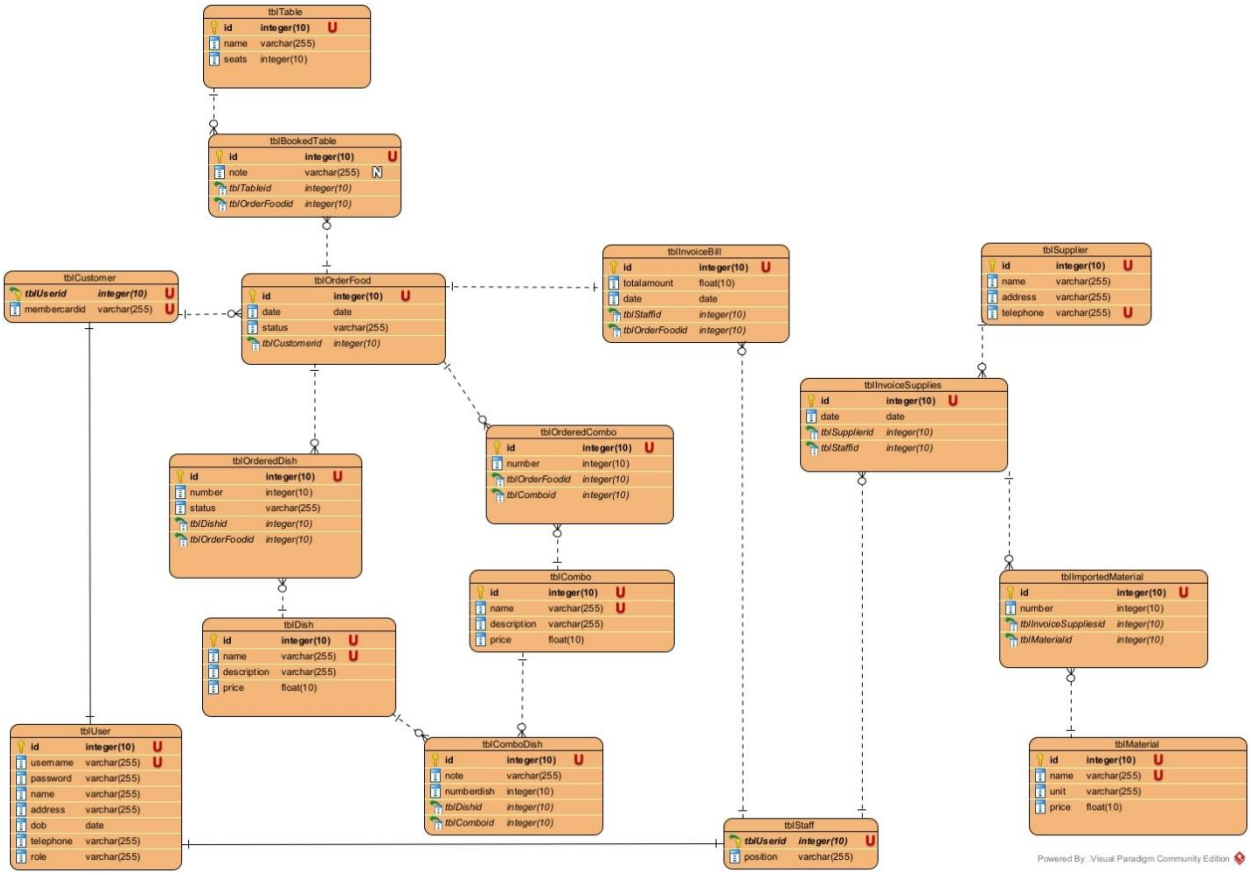
Foreign keys are established for tables:

- 1 tblUser - 1 tblCustomer → table tblCustomer has foreign key tblUserId which also serves as primary key

- 1 tblUser - 1 tblStaff → table tblStaff has foreign key tblUserId which also serves as primary key
- 1 tblCustomer – n tblOrderFood → table tblOrderFood has foreign key tblCustomerId
- 1 tblOrderFood – n tblBookedTable → table tblBookedTable has foreign key tblOrderFoodId
- 1 tblTable – n tblBookedTable → table tblBookedTable has foreign key tblTableId
- 1 tblOrderFood – n tblOrderedDish → table tblOrderedDish has foreign key tblOrderFoodId
- 1 tblDish – n tblOrderedDish → table tblOrderedDish has foreign key tblDishId
- 1 tblOrderFood – n tblOrderedCombo → table tblOrderedCombo has foreign key tblOrderFoodId
- 1 tblCombo – n tblOrderedCombo → table tblOrderedCombo has foreign key tblComboId
- 1 tblCombo – n tblComboDish → table tblComboDish has foreign key tblComboId
- 1 tblDish – n tblComboDish → table tblComboDish has foreign key tblDishId
- 1 tblOrderFood – 1 tblInvoiceBill → table tblInvoiceBill has foreign key tblOrderFoodId
- 1 tblStaff – n tblInvoiceBill → table tblInvoiceBill has foreign key tblStaffId
- 1 tblStaff – n tblInvoiceSupplies → table tblInvoiceSupplies has foreign key tblStaffId
- 1 tblSupplier – n tblInvoiceSupplies → table tblInvoiceSupplies has foreign key tblSupplierId
- 1 tblInvoiceSupplies – n tblImportedMaterial → table tblImportedMaterial has foreign key tblInvoiceSuppliesId
- 1 tblMaterial – n tblImportedMaterial → table tblImportedMaterial has foreign key tblMaterialId

Step 5: Remove all statistical tables and derived attributes

- totalamount in tblOrderFood
- totalamount in tblInvoiceSupplies
- totalamount in tblImportedMaterial
- totalamount in tblOrderedCombo
- price in tblOrderedDish

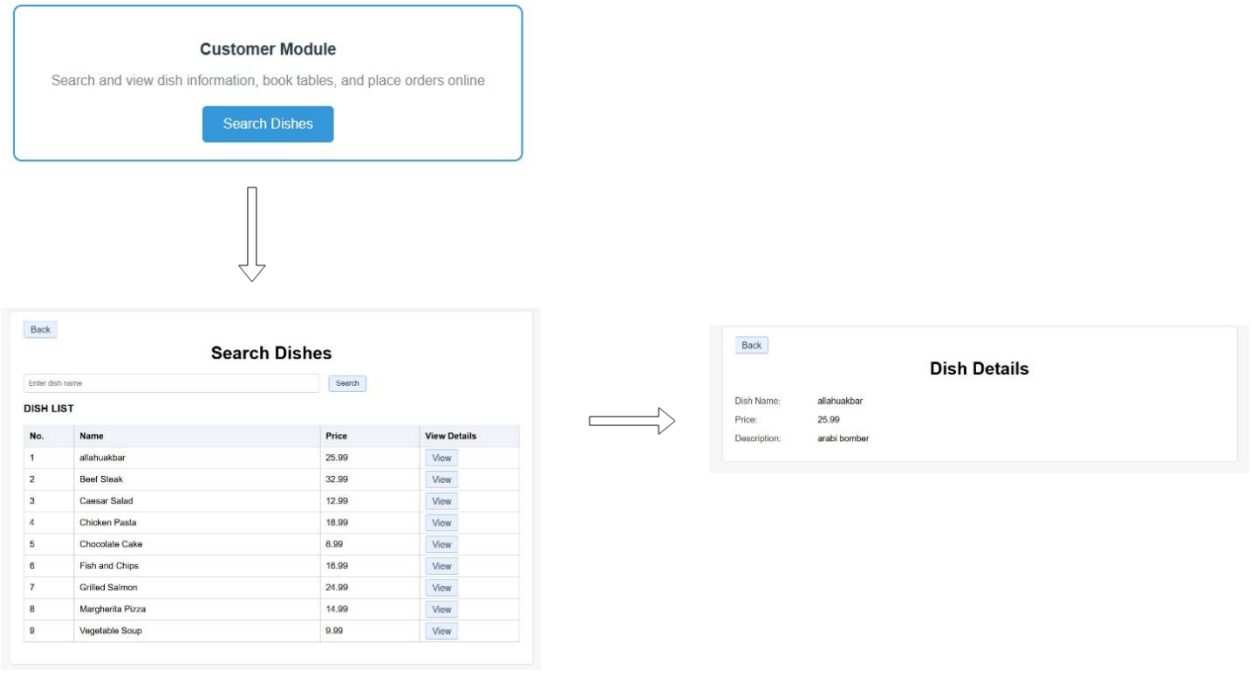


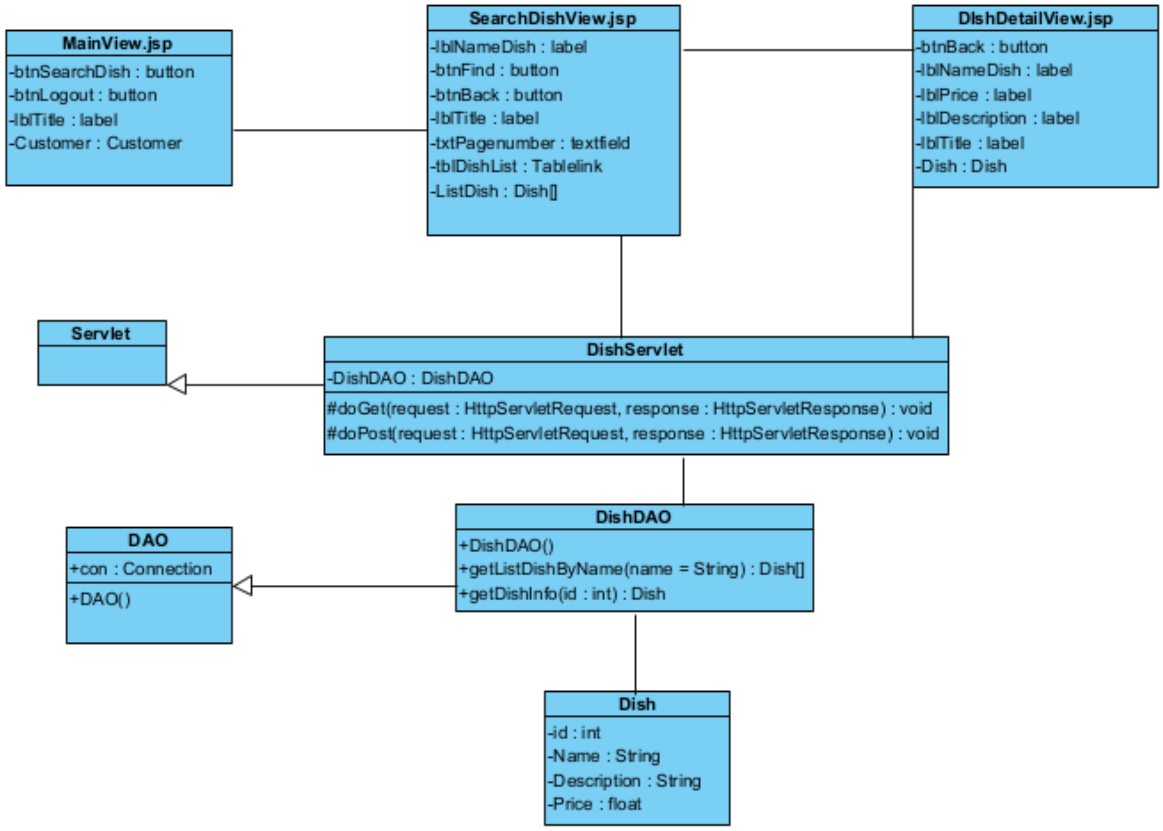
Static Design:

a) Static design for module Search Dish:

- The interfaces are JSP pages: MainView.jsp, SearchDishView.jsp, DIshDetailView.jsp

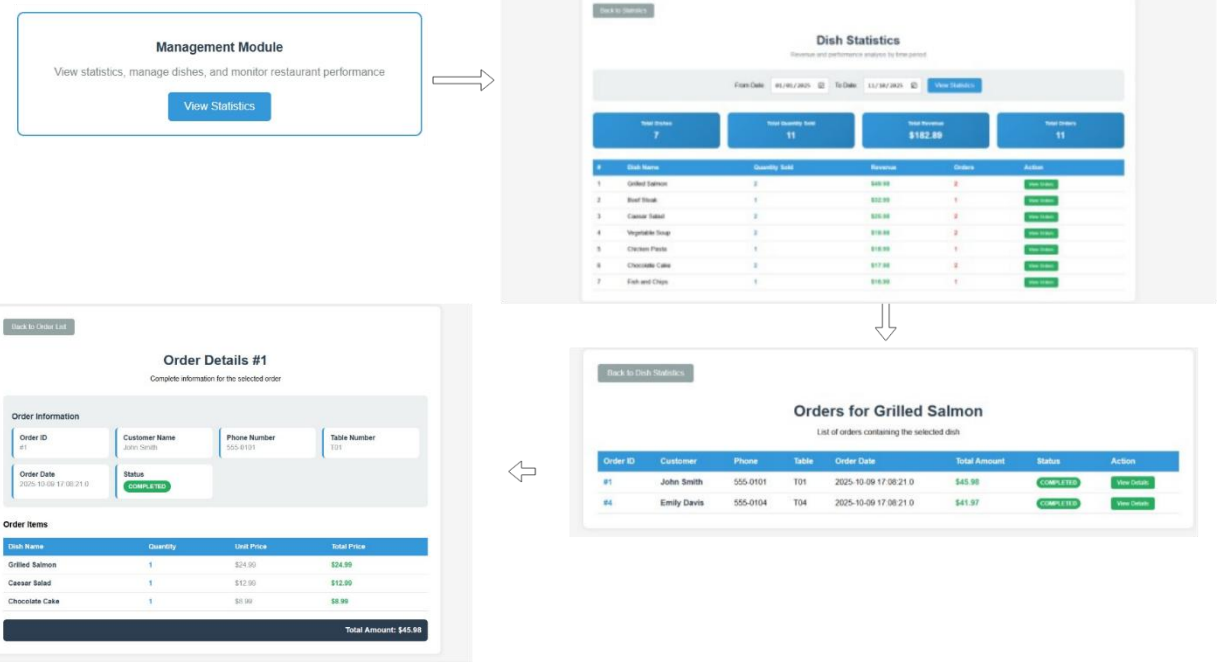
- DataProcessing (Servlet) layer: DishServlet
- The data access layer (DAO) classes: DishDAO
- The related entity classes.

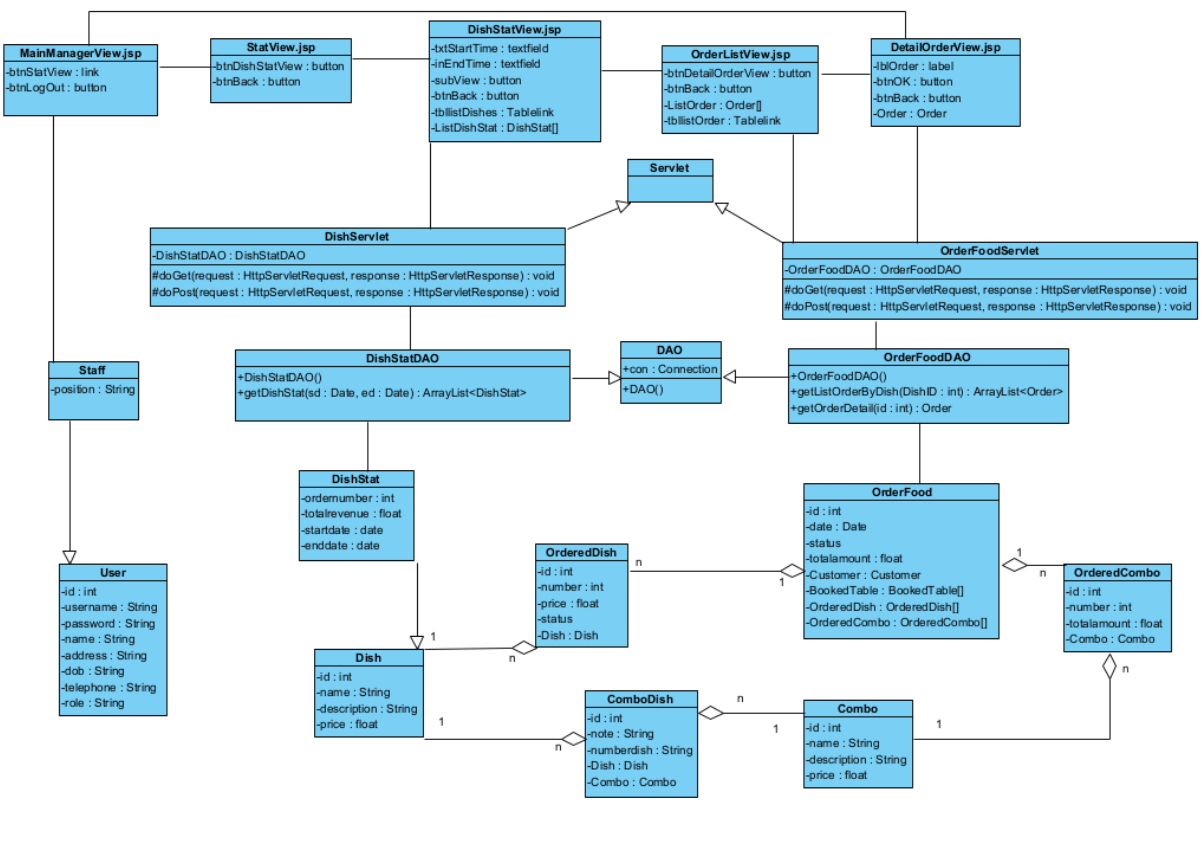




b) Static design for module View Dish Statistic:

- The interfaces are JSP pages: DetailOrderView.jsp, OrderListView.jsp, DishStatView.jsp, StatView.jsp, MainManagerView.jsp.
- The data processing layer (Servlet): OrderFoodServlet, DishServlet
- The data access layer (DAO) classes: OrderFoodDAO, DishStatDAO
- The related entity classes.

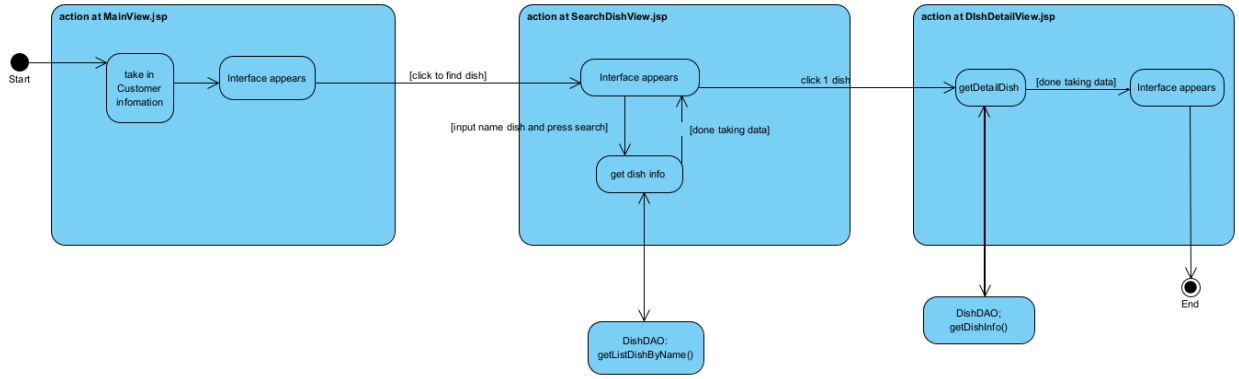

Management Module
View statistics, manage dishes, and monitor restaurant performance
View Statistics
Dish Statistics
Orders for Grilled Salmon
Order Details #1


MainManagerView.jsp
-btnStatView : link
-btnLogOut : button
StatView.jsp
-btnDishStatView : button
-btnBack : button
DishStatView.jsp
-txtStartTime : textfield
-inEndTime : textfield
-subView : button
-btnBack : button
-tbllistDishes : Tablelink
-ListDishStat : DishStat[]
OrderListView.jsp
-btnDetailOrderView : button
-btnBack : button
-ListOrder : Order[]
-tbllistOrder : Tablelink
DetailOrderView.jsp
-lblOrder : label
-btnOK : button
-btnBack : button
-Order : Order
Servlet
DishServlet
-DishStatDAO : DishStatDAO
#doGet(request : HttpServletRequest, response : HttpServletResponse) : void
#doPost(request : HttpServletRequest, response : HttpServletResponse) : void
OrderFoodServlet
-OrderFoodDAO : OrderFoodDAO
#doGet(request : HttpServletRequest, response : HttpServletResponse) : void
#doPost(request : HttpServletRequest, response : HttpServletResponse) : void
Staff
-position : String
DishStatDAO
+DishStatDAO()
+getDishStat(sd : Date, ed : Date) : ArrayList<DishStat>
DAO
+con : Connection
+DAO()
OrderFoodDAO
+OrderFoodDAO()
+getListOrderByDish(DishID : int) : ArrayList<Order>
+getOrderDetail(id : int) : Order
DishStat
-ordernumber : int
-totalrevenue : float
-startdate : date
-enddate : date
OrderFood
-id : int
-date : Date
-status
-totalamount : float
-Customer : Customer
-BookedTable : BookedTable[]
-OrderedDish : OrderedDish[]
-OrderedCombo : OrderedCombo[]
User
-id : int
-username : String
-password : String
-name : String
-address : String
-dob : String
-telephone : String
-role : String
OrderedDish
-id : int
-number : int
-price : float
-status
-Dish : Dish
OrderedCombo
-id : int
-number : int
-totalamount : float
-Combo : Combo
Dish
-id : int
-name : String
-description : String
-price : float
ComboDish
-id : int
-note : String
-numberdish : String
-Dish : Dish
-Combo : Combo
Combo
-id : int
-name : String
-description : String
-price : float

Dynamic design:

a) Dynamic design for module Search Dish

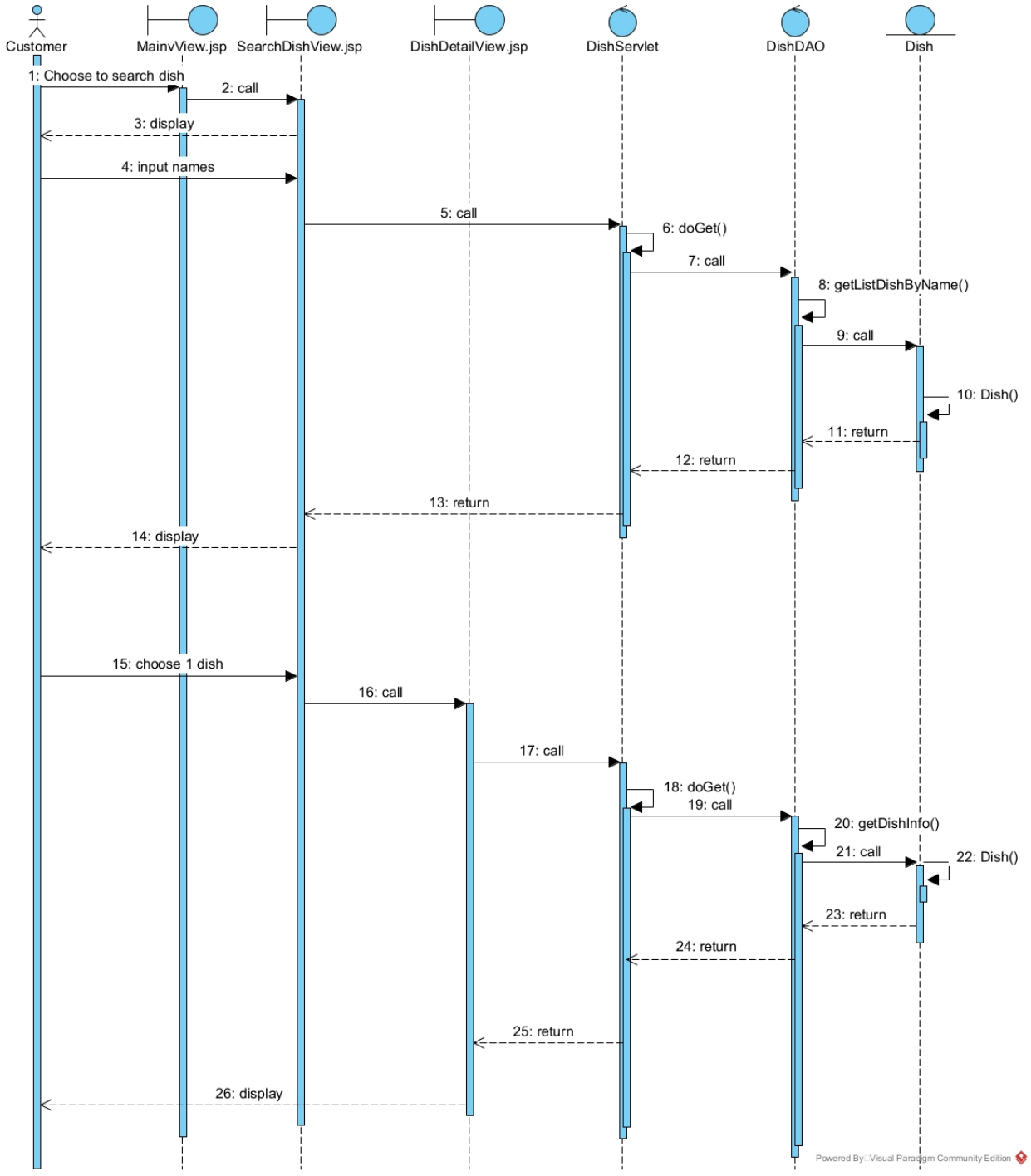
Activity diagram:



Scenario v3:

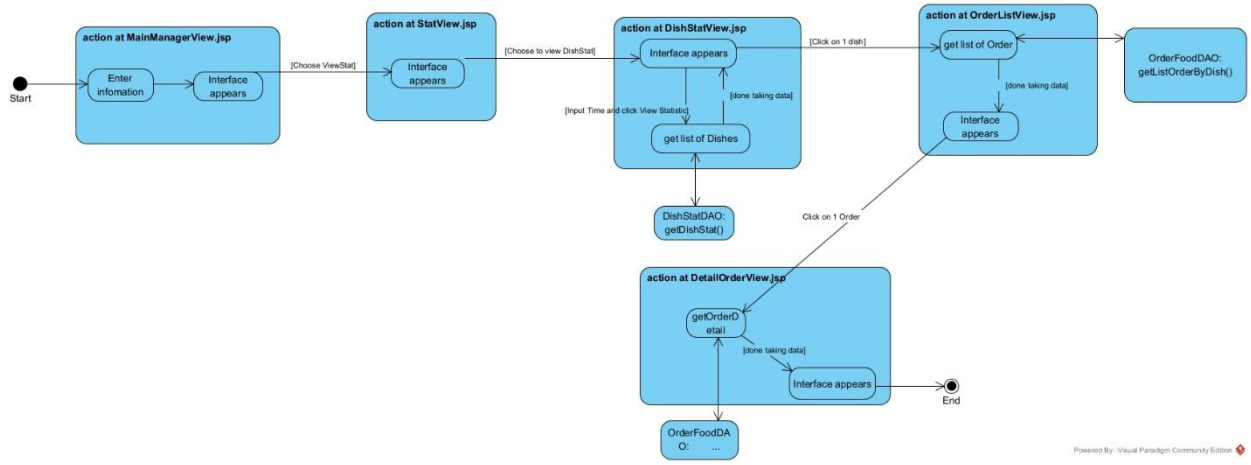
1. On the main customer interface, after logging in, the customer selects the function to search for dish information.
2. The page MainView.jsp calls the page SearchDishView.jsp.
3. The page SearchDishView.jsp is displayed to the customer.
4. The customer enters the dish name and presses search.
5. The page SearchDishView.jsp calls the DishServlet to process the search request.
6. The DishServlet executes the doGet() method.
7. The function doGet() method calls the class DishDAO to request data.
8. The class DishDAO calls the function getListDishByName().
9. The function getListDishByName() executes and calls the class Dish to encapsulate the information.
10. The class Dish encapsulates the entity information.
11. The class Dish returns the result to the function getListDishByName().
12. The function getListDishByName() returns the result to the DishDAO.
13. The DishDAO returns the list of dishes to DishServlet.
14. The DishServlet forwards the data to SearchDishView.jsp.

15. The page SearchDishView.jsp displays the list of dishes to the customer.
16. The customer selects one dish from the list.
17. The page SearchDishView.jsp calls the DishServlet to request details of the selected dish.
18. The DishServlet executes the doGet() method for dish details.
19. The function doGet() method calls the DishDAO to get detailed information.
20. The DishDAO calls the function getDishInfo().
21. The function getDishInfo() executes and calls the Dish class to encapsulate information.
22. The Dish class encapsulates the dish entity information.
23. The Dish returns the result to getDishInfo().
24. The getDishInfo() function returns the result to DishDAO.
25. The DishDAO returns the detailed dish information to DishServlet.
26. The DishServlet forwards the information to DishDetailView.jsp, which then displays the details to the customer.



b) Dynamic design for module View Dish Statistic:

Activity diagram:

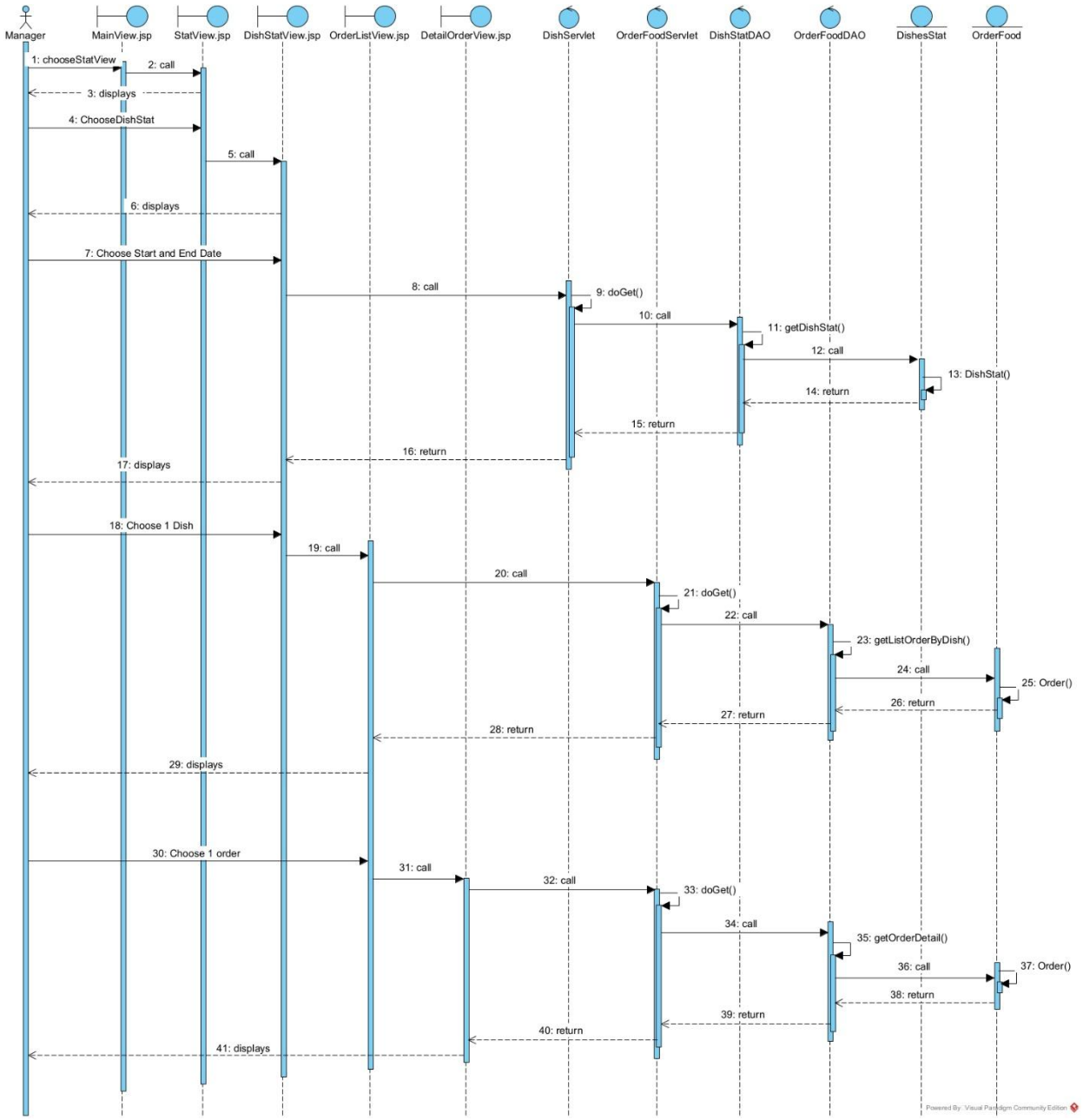


Scenario v3:

1. On the main manager interface, after logging in, the manager selects the “View Statistics” function.
2. The page MainView.jsp calls the page StatView.jsp.
3. The page StatView.jsp is displayed to the manager.
4. The manager chooses “Dish Statistics.”
5. The page StatView.jsp calls the page DishStatView.jsp.
6. The page DishStatView.jsp is displayed to the manager.
7. The manager enters a start date and end date for the statistics.
8. DishStatView.jsp sends a request to DishServlet.
9. DishServlet executes the doGet() method.
10. The doGet() method calls DishStatDAO.
11. DishStatDAO executes the getDishStat() function to retrieve dish statistics from the database.
12. The function getDishStat() creates objects of the DishStat class to encapsulate the retrieved statistics.
13. DishStat encapsulates and returns the statistical data.
14. The function getDishStat() returns results to DishStatDAO.
15. DishStatDAO returns data to DishServlet.
16. DishServlet sends the processed data back to DishStatView.jsp.
17. DishStatView.jsp displays the dish statistics to the manager.

18. The manager selects one dish from the displayed list.
19. DishStatView.jsp calls OrderFoodServlet to view the orders containing that dish.
20. OrderFoodServlet executes the doGet() method.
21. The doGet() method calls OrderFoodDAO.
22. OrderFoodDAO executes the getListOrderByDish() function.
23. The function retrieves all orders that include the selected dish.
24. The function creates OrderFood objects to encapsulate each order's data.
25. OrderFood encapsulates and returns the order data.
26. The function getListOrderByDish() returns the list of orders to OrderFoodDAO.
27. OrderFoodDAO returns results to OrderFoodServlet.
28. OrderFoodServlet sends the processed data back to OrderListView.jsp.
29. OrderListView.jsp displays the list of orders containing the selected dish.
30. The manager selects one order from the list to view details.
31. OrderListView.jsp calls OrderFoodServlet again to retrieve order details.
32. OrderFoodServlet executes the doGet() method.
33. The doGet() method calls OrderFoodDAO.
34. OrderFoodDAO executes the getOrderDetail() function.
35. The function retrieves detailed information about the selected order.
36. The function creates OrderFood objects to encapsulate the detailed information.
37. OrderFood encapsulates and returns the order detail data.
38. The function getOrderDetail() returns results to OrderFoodDAO.
39. OrderFoodDAO returns the data to OrderFoodServlet.
40. OrderFoodServlet forwards the order detail data to DetailOrderView.jsp.
41. DetailOrderView.jsp displays the detailed information of the selected order to the manager.

Sequence diagram:



Package diagram

The entity classes are placed together in the model package.

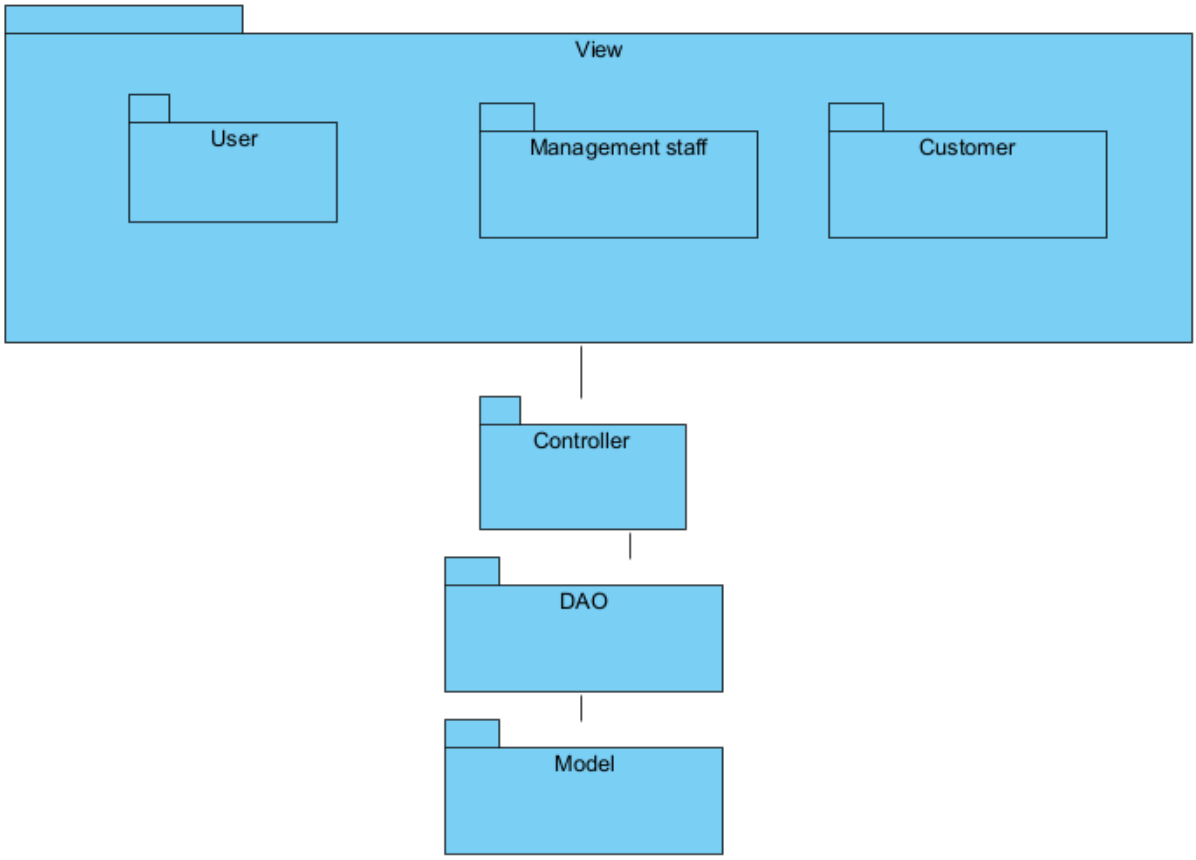
The DAO classes are placed together in the dao package.

The JSP pages are placed in the view package. The view package is further divided into smaller subpackages corresponding to the interfaces for different types of users:

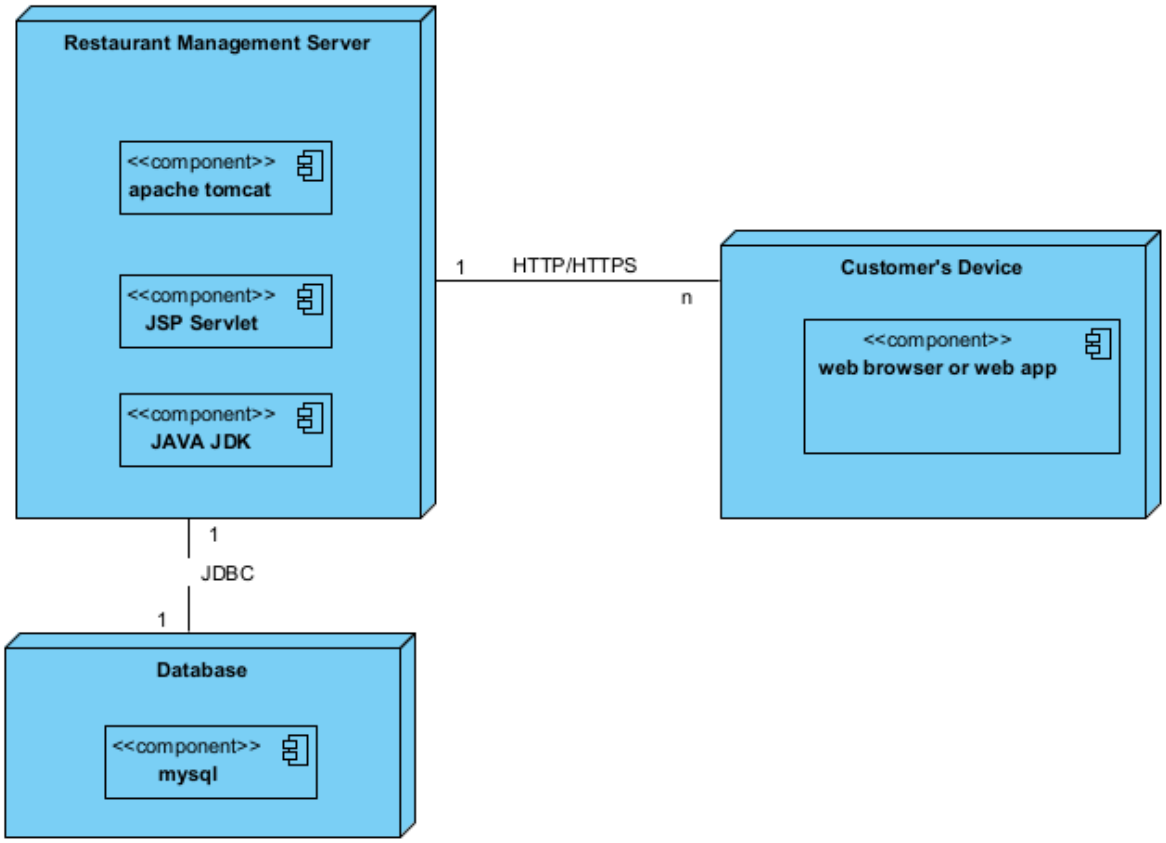
Pages related to login operations are placed in the user package.

Pages related to Management staff functions are placed in the Management staff package.

Pages related to customer functions are placed in the Customer package.



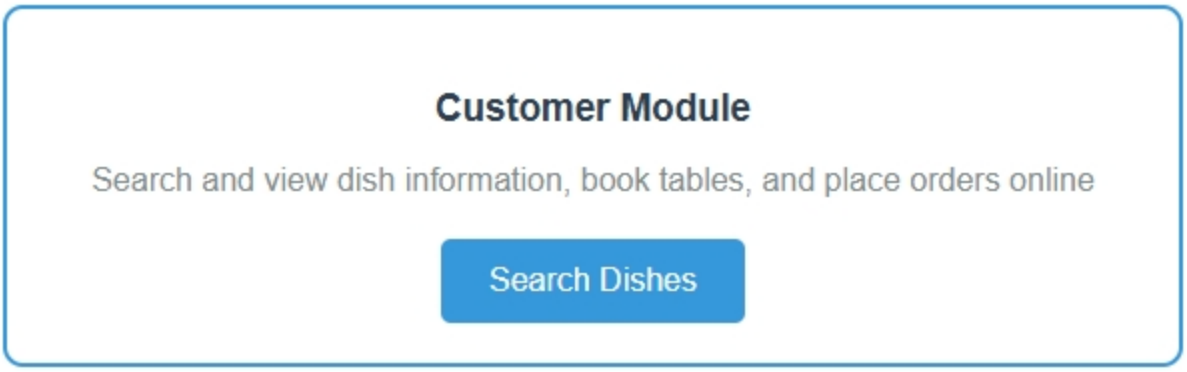
Deployment diagram

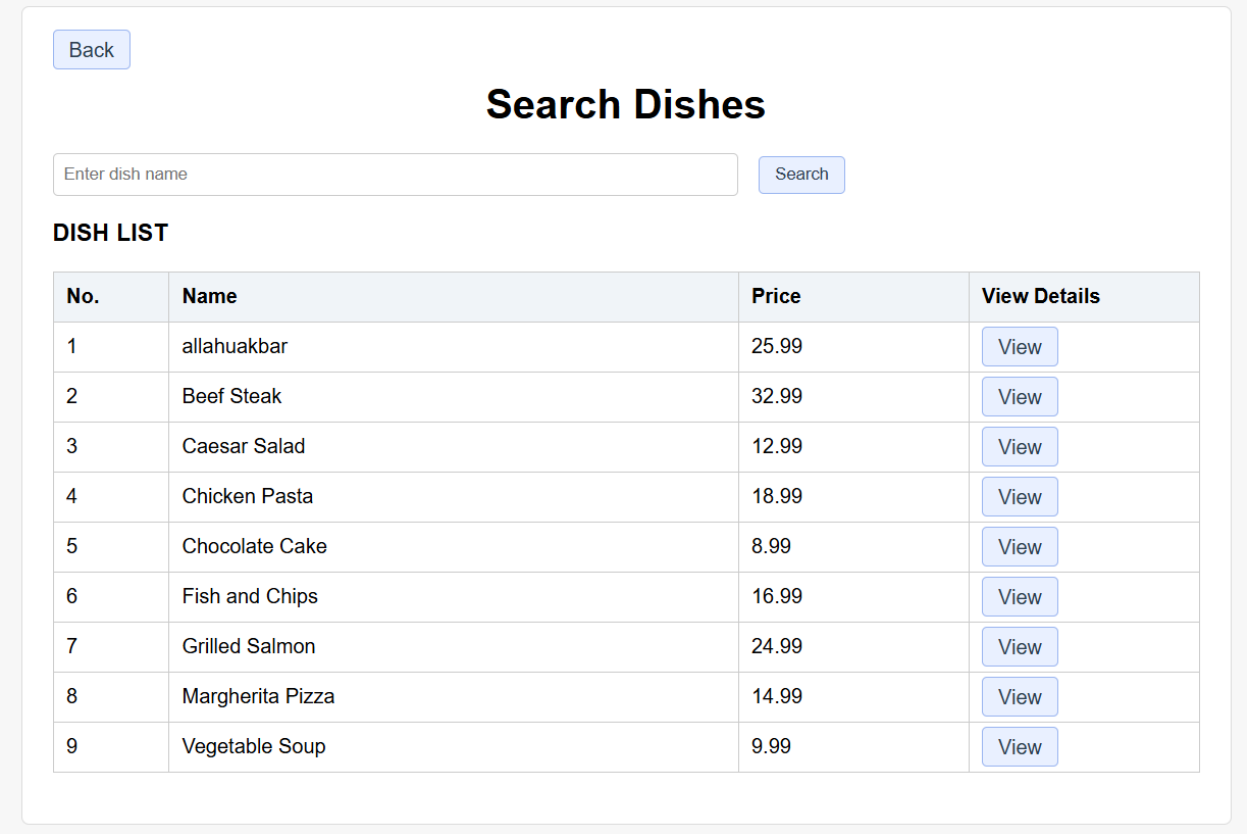


IV. Coding

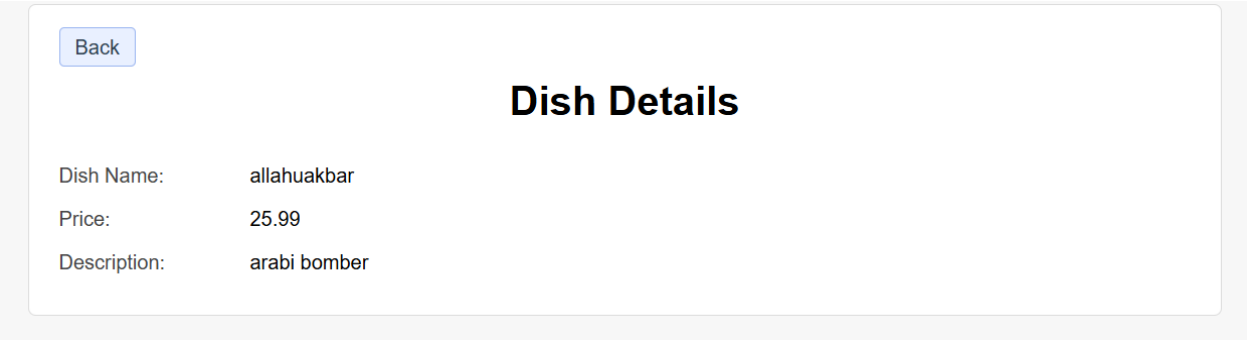
a) Pictures of interface

Module search dish:



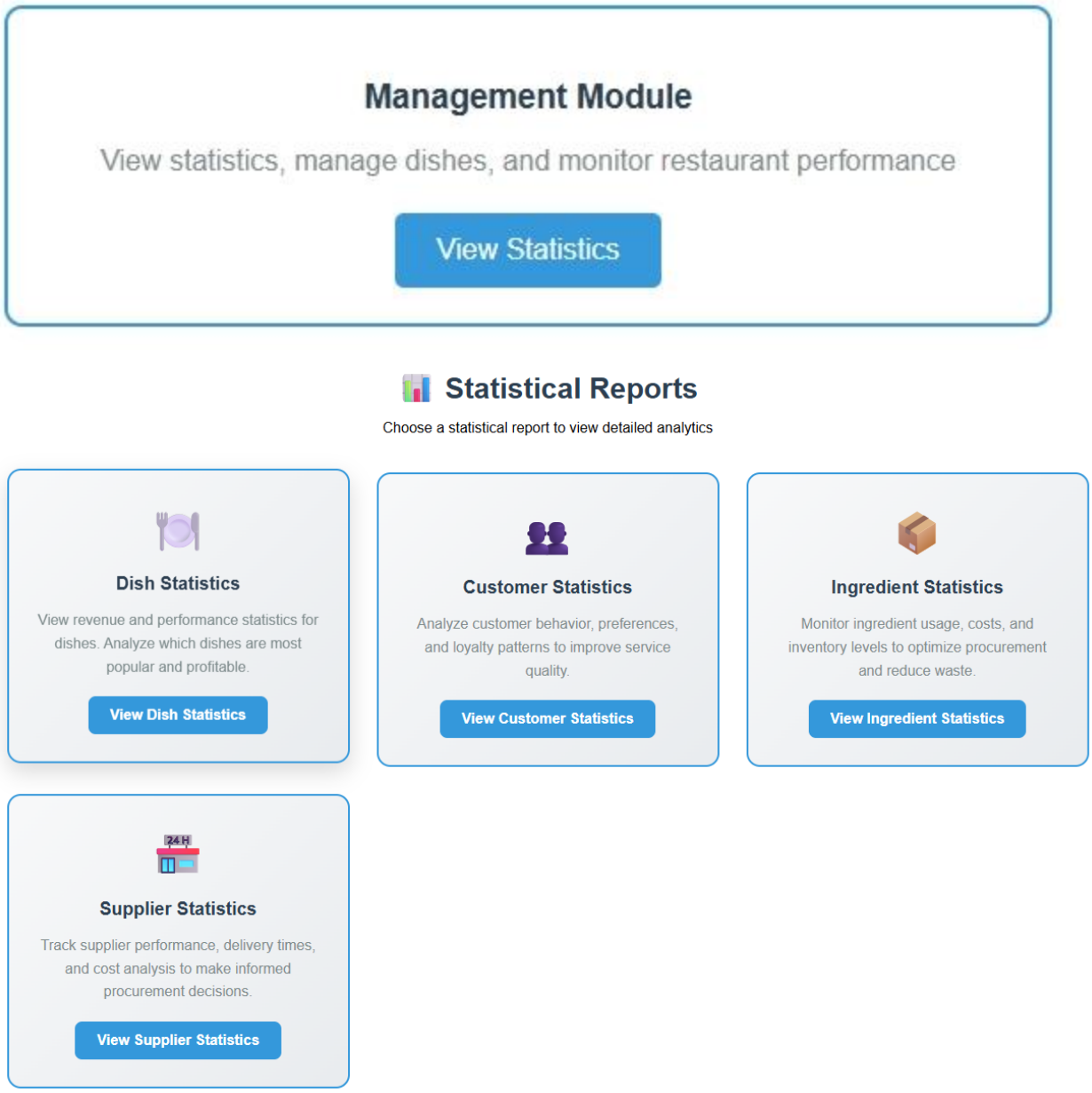


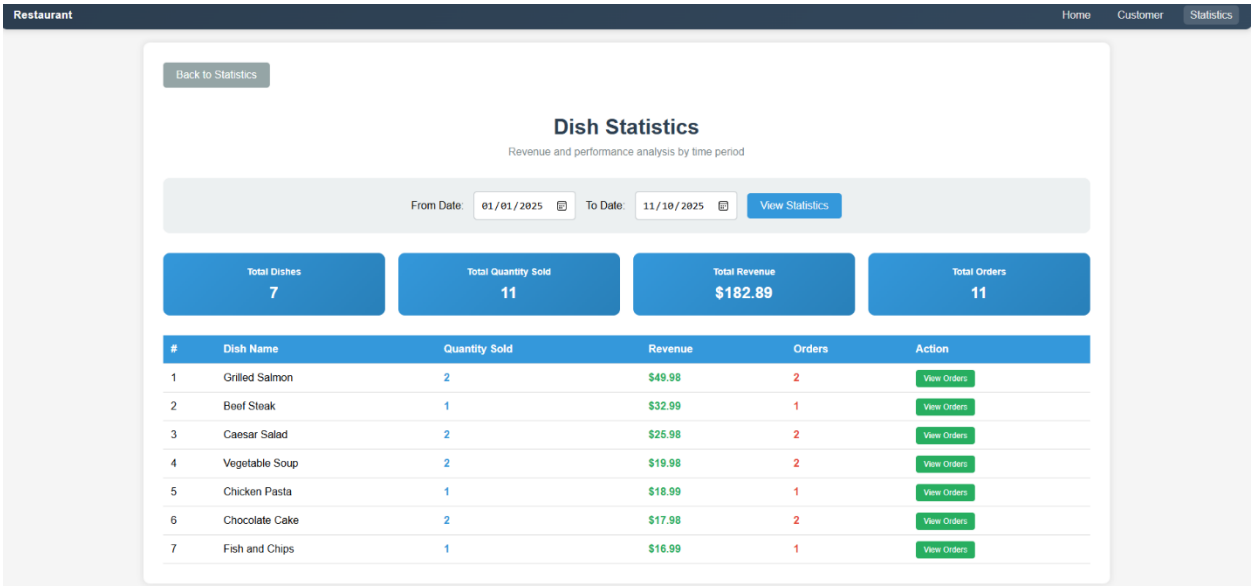
searchDish.jsp



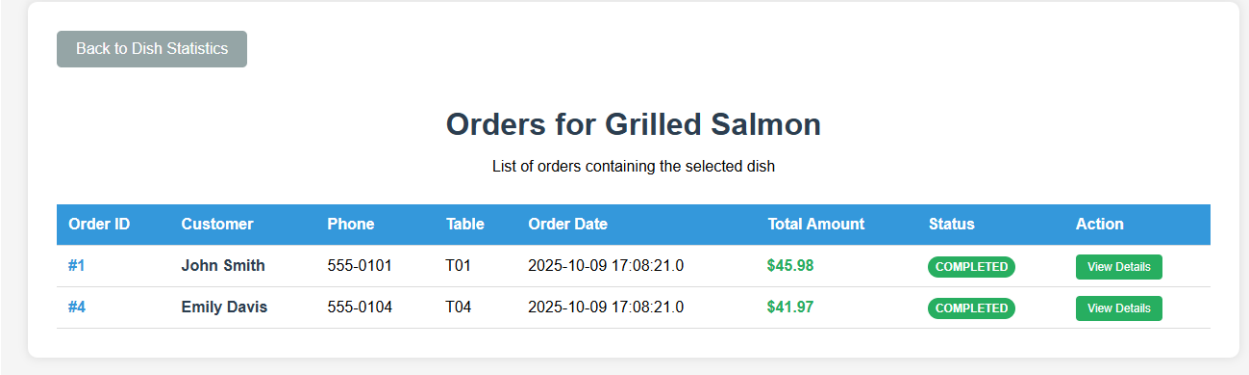
dishDetail.jsp

Module View Dish Statistic

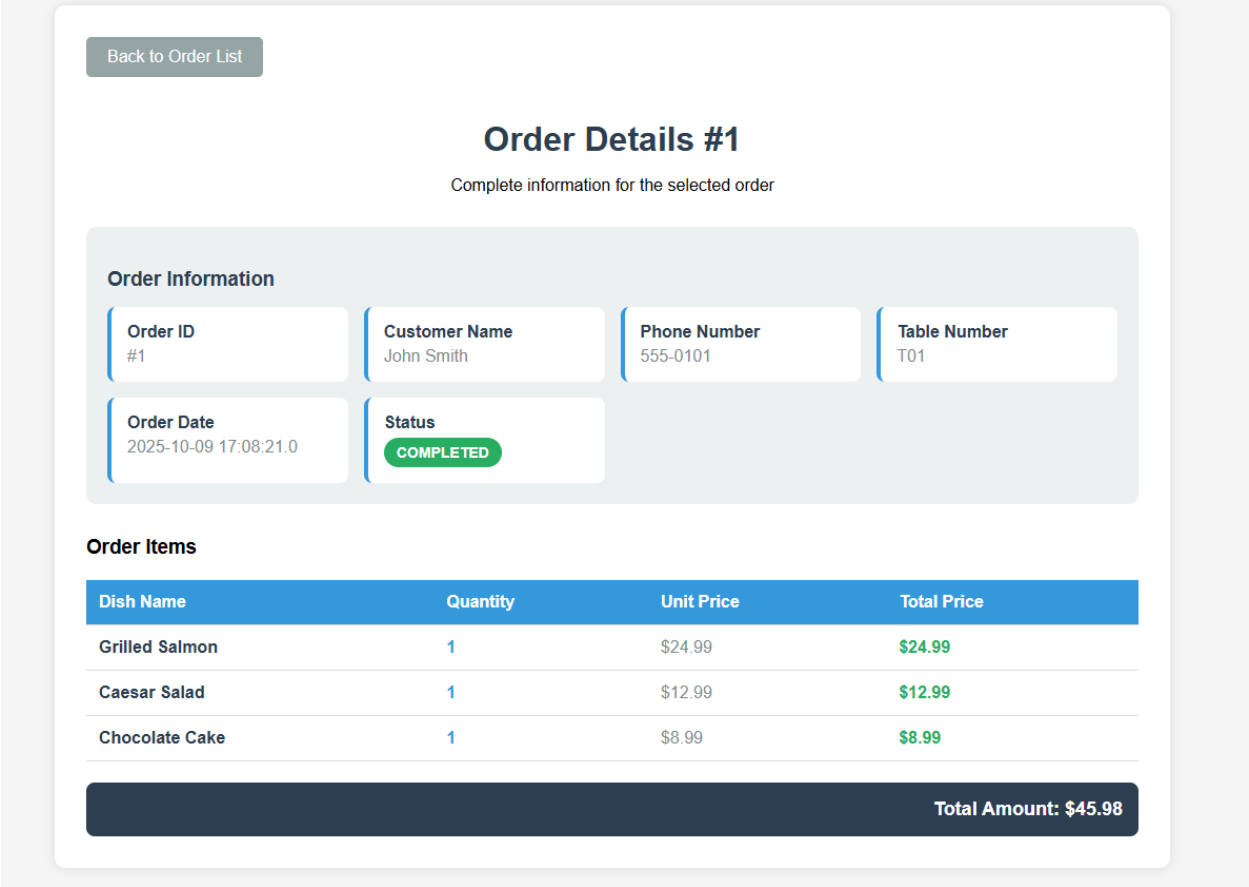




dishStatView.jsp

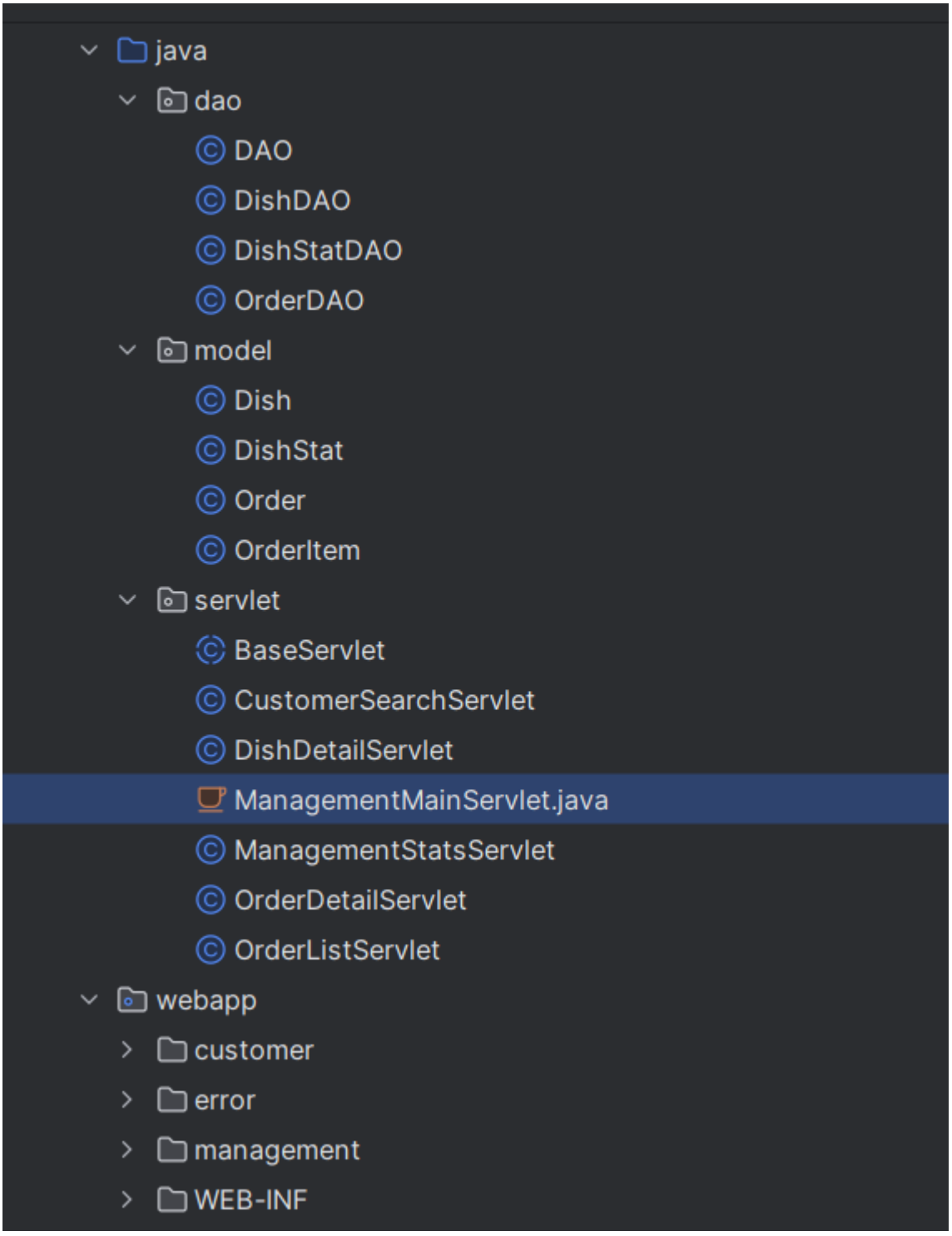


orderListView.jsp



detailOrderView.jsp

b) Code Structure



c) Source code:

https://github.com/allahu-akbar69/manigga-res